CANONICAL HANDBOOK · RELEASE 0.1.0

Converge

A complete, evidence-led method for carrying an idea through human agreement, bounded machine execution, and independently checked settlement.

One descent. Nine pass contracts. One conductor. One barrier.

Converge turns ambiguous intent into durable artifacts, makes the definition of done travel with each task, and constrains an agent to the smallest authority that task earns. Every transition ends in evidence a human or machine can inspect.

Agreement descends

Business intent, observable behavior, architecture, seams, and atomic tasks are resolved in that order.

Authority narrows

Each signed task revision earns only the evidence, tools, paths, time, and external effects it requires.

Evidence accumulates

Gate tokens, hashes, eval output, holdout verdicts, commits, handoffs, and confirmed receipts preserve the real outcome.

RELEASE
Package 0.1.0

EDITION
Canonical · August 2026

SCOPE
Method · Conductor · CLI · Runtime

ORIENTATION · 01

The contract of this handbook

This is a current-state operating reference, not a roadmap presented as reality.

Converge has several kinds of truth, and they are intentionally not interchangeable. This handbook reads the live package in the following order: executable source and tests define semantic boundaries; `cvg help` and `agent-context` establish command existence and syntax; each skill's `SKILL.md` and README define its operating contract; release notes record demonstrated behavior and known gaps; plans describe future work. Earlier PDFs provide visual lineage only.

Version rule. Release `0.1.0` names the whole package: CLI, skills, manifests, and Task-Spec implementation ship together. Internal contract identifiers remain independently versioned where compatibility requires it—for example Task-Spec `format_version: 3`, the agent contract version, and the `hmac-sha256-v2` signing scheme.

The handbook explains what the method promises, what each pass consumes and emits, which checks establish a machine-verifiable claim, where human judgment is still required, and how runtime authority is granted and revoked. A green gate never proves more than its stated contract. A tracker is a projection, not the backlog. The conductor can sequence work but cannot declare it correct. A model's confident prose is not a receipt. A future Manager is not silently implied by the single-task Loop that exists today.

Examples use the canonical `cvg/` workspace. Under `cvg/docs/`, each artifact type owns a folder: `brd/`, `tech-spec/`, `no-go/`, `adrs/`, and `lessons/`. Legacy flat prefixes and bare `docs/`, `sketch/`, and `tasks/` layouts remain supported fallbacks. Explicit paths always win.

CLAIM TYPE	REQUIRED EVIDENCE	INSUFFICIENT SUBSTITUTE
Method contract	Current skill instructions, executable source, and tests	An earlier handbook or help summary alone
Next pass in the route	`cvg next` plus workspace evidence and the selected lane	Chat memory or a pass number guessed from the transcript
Command exists	Current `cvg help` or `agent-context`	A row in `PLAN.md`
Task may run unattended	Exact ready-frontier selection, Tier 1, current Pass 7 profile, and separately reviewed host-control evidence	A valid-looking YAML file or Bind PASS alone
Ready for lifecycle close	Stamped `accepted: true`, nonempty accepted-by/at, and a parseable matching `task_id` / `result: pass` receipt	Settlement evidence, success token, or tracker status alone

`cvg bind --supervised` can mint a Tier-2 profile, but the profile records only the waiver: release 0.1.0 does not prove a supervisor is present when Loop launches.

When sources disagree, stop at the narrowest claim that can be verified. Correct the owning source before expanding the claim. This rule is especially important for generated indexes, tracker projections, cached documentation, and runtime logs: each may be useful evidence, but none may silently outrank the canonical artifact from which it derives.

ORIENTATION · 02

Contents — five views of one system

Read linearly for the full descent, or enter through the layer you operate.

If you own the outcome

Start with the two-phase architecture, then read Passes 0–4 and the barrier. Your key decisions are the no-go option, acceptance metrics, irreversible architecture, resolved objections, and final owner sign-off.

If you operate agents

Start with the conductor, then Tasking, Bind, and the Loop. Pay special attention to the selected lane, prompt ownership, Task-Spec signatures, capability closure, fresh attempts, stop states, and tier-2 independent verification.

If you integrate tooling

Read the CLI, workspace resolver, JSON envelope, adapter boundaries, and shipped-versus-planned ledger before building automation around commands.

If you audit evidence

Follow artifact lineage and receipts. Verify hashes, stable tokens, path policy, evaluation output, tier-2 verdicts, and the distinction between a local commit and a pushed branch with auto-or-manual PR handoff.

Operating shortcut. For any question, locate the owning altitude first. Product meaning belongs to the BRD or technical specification; irreversible system choices belong to ADRs; ordering and seams belong to plans; delegated behavior and evals belong to the Task-Spec; runtime rights belong to the execution profile; actual outcomes belong to Loop logs, terminal state, verifier output, and the receipt's recorded subset. Read adjacent artifacts for traceability, but edit only the owner and deliberately renew every downstream proof.

The handbook does not replace each skill's invocation-specific instructions or the CLI's generated command manifest. It supplies the complete architectural map and the reasoning needed to interpret those detailed surfaces correctly. Commands shown here should always be confirmed against the installed `0.1.0` package before automation is committed.

FOUNDATIONS · 03

The thesis: reduce ambiguity before spending autonomy

Converge is a controlled descent from owner intent to executable proof.

Software work fails when decisions change altitude without becoming explicit. A business outcome becomes an implementation preference; a plan becomes a task before its seams are understood; an agent is handed a broad repository and told to "make it work." Converge inserts durable contracts at each altitude so uncertainty is resolved where it is cheapest.

01 **Intent becomes falsifiable.** Outcomes, scope, constraints, and acceptance metrics are written before architecture or implementation.

02 **Structure becomes durable.** Hard-to-reverse decisions and shared vocabulary are grounded against the live system.

03 **Work becomes decomposable.** Natural seams become swimlanes; lanes become independently provable legs.

04 **Plans are attacked before automation.** A different model family tries to refute them, and an owner resolves or accepts every objection.

05 **Execution becomes bounded.** Each leaf task carries its own evals, write scope, budgets, and stop conditions; the runtime binds only the evidence and authority that revision earns.

06 **Completion becomes evidence.** Green task evals, path checks, confirmed settlement receipts, and an optional Tier-2 verdict replace confidence with inspectable proof.

The method is not "more documents." Its unit of value is a *closed transition*: a named input is transformed into a named artifact, and a gate states exactly what was checked. Human judgment remains at semantic boundaries; deterministic scripts enforce structural and runtime invariants. This division prevents two opposite errors—automating decisions that require an owner, and asking a human to repeatedly verify facts a machine can prove.

Human authority

The owner decides whether the problem is worth solving, resolves material product trade-offs, accepts residual risk, and signs the Pass 4 barrier. A gate may expose missing evidence; it cannot impersonate that decision.

Machine authority

Scripts discover files, lint contracts, recompute hashes, check graph and path invariants, run evals, and emit stable verdicts. Agents may research, draft, attack, or implement only inside the artifact and capability boundary of their seat.

Operationally, this means "go faster" is achieved by shrinking the error surface, not deleting controls. A FAST lane reuses already settled intent and architecture; it still authors, signs, binds, evaluates, and settles a leaf. Conversely, a FULL lane is not automatically safer if its owner approvals are ceremonial or its evals observe the wrong behavior. The quality of each closed transition matters more than the number of pages it produces.

FOUNDATIONS · 04

The architecture: six trust layers and one conductor

The conductor keeps the route in order; artifacts, gates, runtimes, and evidence keep every claim honest.

Converge is deliberately split so no single layer can silently redefine the others. Human intent enters through durable artifacts. The conductor reads those artifacts to determine the next pass and supply its canonical prompt. Pass skills describe the transformation. The `cvg` referee routes to proven scripts and exposes stable verdicts. Runtime contracts narrow the authority available to an external engine. Logs, verdicts, terminal state, and receipts preserve complementary parts of what happened.

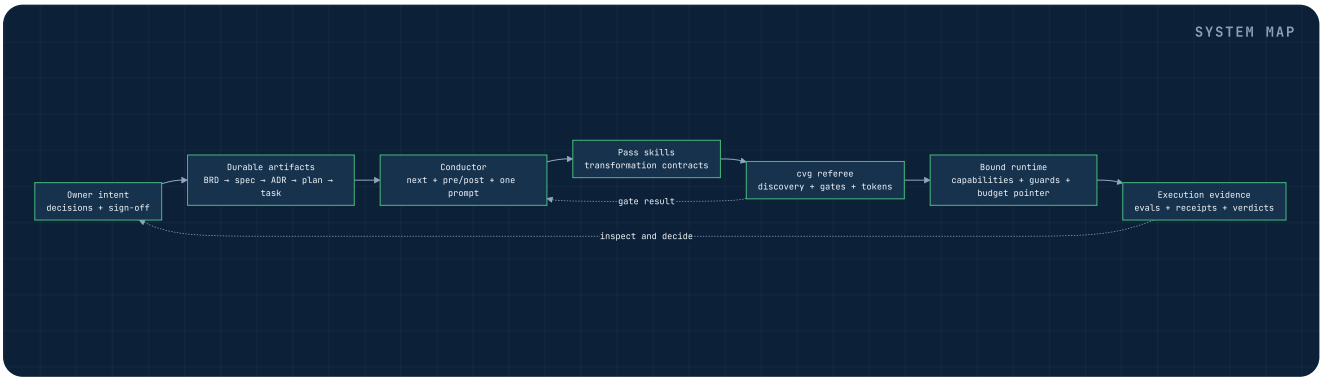
LAYER	OWNS	MUST NOT PRETEND TO OWN
Owner	Priorities, trade-offs, sign-off, accepted risk	Mechanical validation
Artifacts	BRD, spec, ADRs, plans, Task-Specs, profiles	Runtime enforcement by declaration alone
Conductor	Lane-aware order, evidence board, prompt handoff, pre/post presence checks	Passing a gate or choosing a lane
Skills	Pass workflow, questions, transformations, gates	Fleet scheduling
CLI/referee	Discovery, routing, stable tokens, adapters	Rewriting proven pass logic
Runtime	Task-scoped tools, paths, fresh attempts, and brakes derived from Task-Spec budgets	Changing signed routing or budget authority
Evidence	Evals, hashes, logs, receipts, independent verdicts	Replacing semantic owner review

This separation also explains portability. Claude, Codex, Kimi, or another backend may occupy the worker or judge seat, but the task contract, gate semantics, path policy, and receipts remain Converge artifacts. Vendors enter through flags and adapter manifests; they are not baked into the method.

Layer failure rule. Artifacts may declare policy, but only a runtime can enforce it. A runtime may execute, but only signed artifacts may authorize it. Evals may prove declared behavior, but a different-family holdout may still expose Goodharting. Receipts may attest an execution, but they cannot rewrite its input contract.

Ownership also determines recovery. A stale hash is repaired by regenerating the derived profile or review stamp; a missing business decision returns to the owner; an unavailable enforcement primitive requires a different runtime or an explicit policy waiver; a tracker mismatch is rebuilt from canonical specs. Keeping those repairs in their own layer prevents the most dangerous shortcut: making a downstream tool edit upstream truth merely to turn its own check green.

The conductor is deliberately read-only. It can refuse Pass N when required evidence from an earlier lane pass is absent, but it cannot grant a PASS. The CLI sits between portable method and external systems. It normalizes discovery and tokens, while model engines and tracker adapters retain their own authentication. This keeps the referee auditable and limits credential spread.



FOUNDATIONS · 05

Two phases, one barrier, nine passes

Human-led design converges first; machine-led build begins only after Consensus closes.

The numbered method runs from Pass 0 through Pass 8. Pass 0 is optional when a usable brief already exists. Pass 6 is opt-in when an external tracker is useful. Those exceptions change the route, not the meaning of any gate. Pass 4 is the singular barrier: hardened plans and the objection log require owner sign-off before downstream machine execution.

Design · 0–4

Capture, Intent, Structure, Decompose, Consensus. Humans own meaning; agents research, challenge, and draft.

Barrier · 4

Different-family refutation, provenance-stamped objections, complete resolution, explicit owner approval.

Build · 5–8

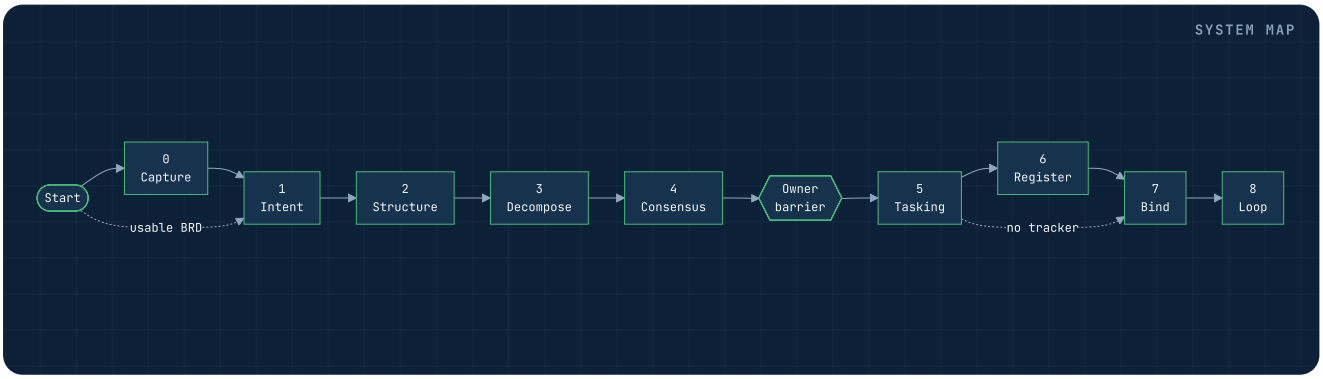
Tasking, optional Register, Bind, Loop. Machines operate inside signed, hash-bound, testable contracts.

The package contains 12 skills. Nine form the numbered spine: `idea-to-brd`, `brd-docs-to-tech-req`, `tech-req-to-adrs`, `reqs-to-swimlane-plans`, `sketch-plans-adversarial-review`, `task-spec`, `task-specs-to-issues`, `task-to-runtime-contract`, and `task-loop`. Three are utilities: `conductor-steps` walks the selected route and owns the canonical pass prompts; `pass-to-lesson` turns a closed pass into a teachable lesson; and `skill-creator` authors, evaluates, packages, and validates skills. Utilities do not renumber the chain.

The arrow is not irreversible. A downstream contradiction must route back to the pass that owns it. A missing ADR is not patched inside the Loop; a broken eval is not "worked around" by the worker; a new product decision is not hidden inside a task brief. Returning upstream preserves causality and keeps the durable record aligned with reality.

BOUNDARY	AUTHORITY AT EXIT	PROOF CARRIED FORWARD
Passes 0–1	Owner approves problem, outcome, scope, and acceptance	BRD and falsifiable technical specification
Passes 2–3	Engineering grounds irreversible choices and natural seams	ADRs, vocabulary, swimlanes, legs, proving tests
Pass 4	Owner accepts hardened plans and visible residual risk	Fresh objection log plus sign-off
Passes 5–7	Tier-1 task defines authority; static Bind derives controls; host readiness is separately probed	Spec, hashes, profile, guards, brief, optional host-doctor evidence
Pass 8	Policy allows only the terminal effect supported by evidence	Eval, verify verdict, commit/PR or explicit handoff, plus an attempted receipt that must be confirmed before lifecycle close

A pass can be revisited, but it cannot be skipped retroactively. The conductor makes that order visible from workspace evidence; the gates still decide whether the evidence is valid. Any edit to an upstream decision invalidates the dependent review, signature, profile, or receipt claim until the affected gates are renewed.



FOUNDATIONS · 06

Routes choose ceremony; they never waive proof

The router recommends FAST, NORMAL, or FULL without becoming a pass or a scheduler.

`cvg lane <intent>` is an offline, deterministic classifier. It evaluates the kind of change, expected file surface, and risk markers, then prints one route token. It does not execute the route, choose a task, or relax a gate. Every route still requires a signed TaskSpec before unattended work.

Once the route is chosen, pass it to `cvg next --lane ...`. The router answers *which passes apply*; the conductor answers *which of those passes comes next*. Keeping those decisions separate prevents a sequence tool from quietly lowering risk.

LANE	PASS SHAPE	APPROPRIATE WHEN
FAST	5 → 7 → 8	A small, reversible change fits an existing seam and established architecture.
NORMAL	1 → 2 → 5 → 7 → 8	Intent or grounding needs clarification, but new multi-lane planning is unnecessary.
FULL	0 → 8	A new system/seam forces the complete descent; file-count tightening raises an already-NORMAL result above 15 files to FULL.

Risk overrides convenience. Authentication, money movement, migrations, secrets, public API changes, irreversible operations, and work that creates a new architectural seam cannot ride FAST. Sensitive or irreversible markers floor the classifier at NORMAL; a new system/seam or the larger file threshold raises it to FULL. Pass 6 remains an opt-in projection choice; the canonical task queue can remain repo-local. The classifier emits `LANE=FAST|NORMAL|FULL|USAGE_ERROR`, which automation can parse without interpreting prose.

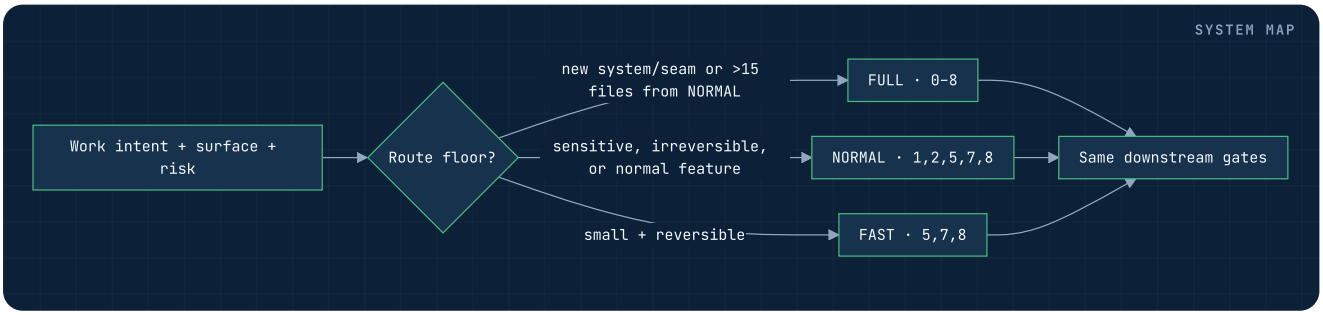
File-count tightening is applied once: more than five files raises FAST to NORMAL, while more than fifteen raises an already-NORMAL result to FULL. Because those branches are mutually exclusive in 0.1.0, a trivial FAST-shaped intent with sixteen files becomes NORMAL in that single classification rather than cascading twice. Treat the emitted route as current executable truth.

Important boundary: routing and conducted sequencing are shipped; fleet dispatch is not. A future Manager may select ready tasks and coordinate concurrent loops. In the current package, a human or CI supplies the one issue to Pass 8.

```
cvg lane "correct a typo in an existing help string" --files 1
# LANE=FAST

cvg lane "change authentication and migrate stored credentials" --files 6
# LANE=NORMAL

cvg lane "introduce a new authentication service seam" --files 6
# LANE=FULL
```

The classifier examines intent and declared surface, not repository history or a hidden network service. A malformed file count, missing intent, or invalid flag ends in `LANE=USAGE_ERROR`. A recommendation should be reconsidered when later discovery reveals a public contract, secret, money, migration, irreversible effect, or new seam. Escalating ceremony is always allowed; downgrading must be justified by settled upstream evidence.

Register is orthogonal to risk. A FAST task may still be projected when a team needs tracker coordination, while a FULL proof-of-concept may remain repo-local. The route determines method passes; the projection decision determines where operational state is mirrored.



FOUNDATIONS · 07

The conductor: one calm next step at a time

It reads the floor, hands over one canonical prompt, and refuses sequence drift without pretending to be a gate.

The nine pass skills already knew how to transform and close their own artifacts. What was missing was the choreography between them. In a long chat, the current pass could live only in scrollback or session memory. The `conductor-steps` utility replaces that fragile memory with a read-only view of the workspace.

01 **Select the route.** A human accepts a `FULL`, `NORMAL`, or `FAST` lane from `cvg lane`. The conductor never infers or downgrades it.

02 **Read the evidence board.** `cvg next --lane ...` checks the known artifact home for every lane pass and returns `NEXT_PASS=N` or `DONE`.

03 **Open the door.** Before Pass N, `next-pass.sh pre N` refuses when any earlier lane artifact is absent and names the missing pass.

04 **Steer with one prompt.** The agent reads `cvg/brain/_prompts/pass-N-*.md`, not the whole method or an improvised prompt reconstructed from memory.

05 **Leave the artifact in its home.** The pass skill performs its bounded transformation under the typed `cvg/docs/*` folders, `cvg/sketch`, `cvg/tasks`, `cvg/execution`, or `cvg/receipts`.

06 **Check presence, then truth.** `next-pass.sh post N` confirms the artifact landed; the pass's own `cvg` gate gives the authoritative verdict.

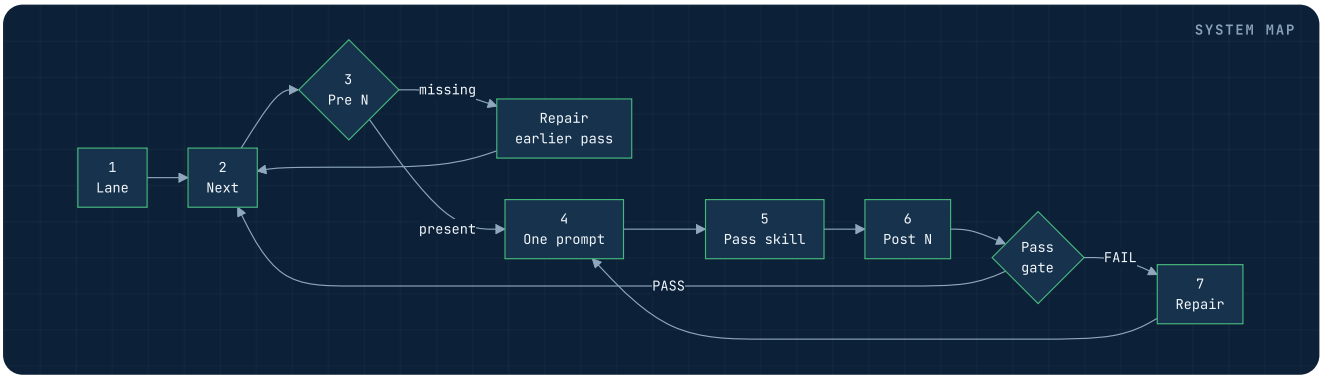
07 **Advance or repair.** A green gate returns to `cvg next`. A failed gate stays at the owning altitude until the canonical artifact is repaired.

THE INVARIANT

Evidence presence is not a verdict. A BRD file can move the evidence board, but only `cvg capture` can say whether that BRD passes. The conductor can refuse forward motion; it can never grant it.

SIGNAL	WHAT IT PROVES	WHAT IT DOES NOT PROVE
`NEXT_PASS=4`	Earlier lane artifacts are present and Consensus is next	Those artifacts passed their gates
`CONDUCTOR_PRE=OK`	Prior lane evidence exists	Pass N is semantically authorized
`CONDUCTOR_POST=OK`	Pass N left its expected artifact	The artifact is correct or accepted

`cvg init` seeds all nine canonical prompts. Pass 6 remains informational and opt-in, so its absence never blocks a route. The conductor writes no state file, edits no artifact, chooses no work item, and owns no credentials. A resumed session can therefore recover position from the same evidence any other session sees.



FOUNDATIONS · 08

Artifact altitude: one decision, one durable home

Each artifact answers a different question and should not leak into the altitude below it.

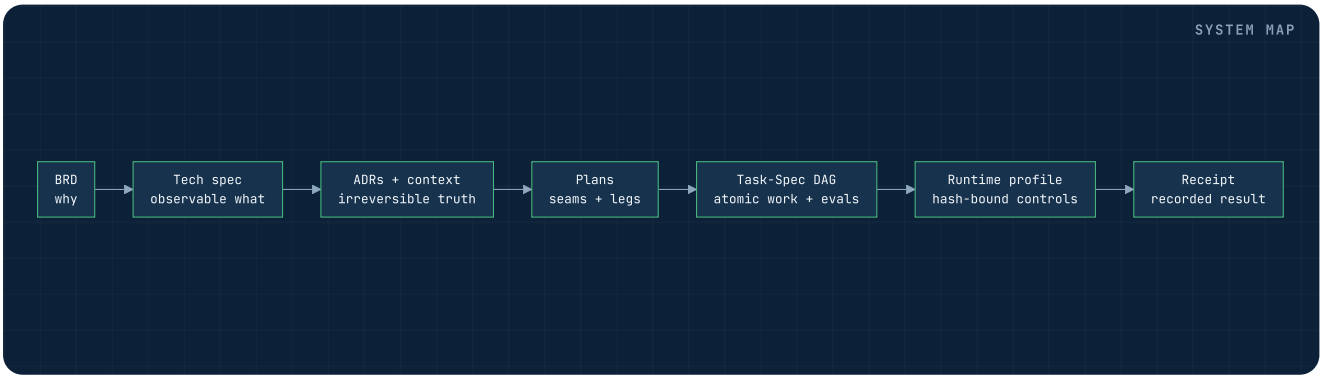
Converge prevents decision drift by assigning every concern to an artifact altitude. The BRD explains why the outcome matters. The technical requirements specifies observable system behavior. ADRs record hard-to-reverse grounding decisions. Swimlane plans cut along real seams without prescribing code. Task-Specs define atomic work and runnable proof. Runtime profiles bind one signed revision to evidence and authority. Receipts record a bounded subset of the execution evidence.

ARTIFACT	PRIMARY QUESTION	WRONG CONTENT AT THIS ALTITUDE
BRD	Why act, for whom, and what is success?	Architecture and file-level implementation
Tech spec	What must the system observably do?	Unfalsifiable aspiration
ADR + context	What is true and difficult to reverse?	A build plan disguised as a decision
Swimlane plan	Where are the seams and proving legs?	Atomic tasks or code snippets
Task-Spec	What bounded leaf can be delegated and proven?	New product or architecture decisions
Execution profile	What may this revision read, write, call, and spend?	Replacement intent
Receipt	What result, bound artifacts, eval evidence, runtime, and branch were recorded?	Predictions or evidence fields the current schema does not contain

Traceability flows downward; correction flows to the owner artifact. This is why the worker brief contains identifiers and pointers instead of copied doctrine. Copying creates a second, drifting source. Hash-bound references keep the chain inspectable without inflating every prompt.

Change propagation. Editing a KPI may require renewed acceptance criteria; editing a requirement may require new ADR or plan review; editing an ADR can invalidate every inheriting lane; editing a plan after attack invalidates its objection-log hash. In Task-Spec, body or v2-covered authorization edits invalidate HMAC, while *any* byte edit invalidates an already-bound whole-file epoch. The break is intentional evidence.

Ownership must also be singular. A tracker issue may quote the Task-Spec but cannot become a second editable goal. `AGENTS.task.md` may point at an ADR but cannot paraphrase it as new doctrine. The current execution receipt records result, artifact/eval hashes, runtime, and branch, but cannot mark task frontmatter accepted. Derived surfaces should expose backlinks and hashes so an auditor can walk upward to the authoritative decision and downward to the resulting proof.



FOUNDATIONS · 09

Workspace resolution is part of correctness

Converge resolves against the project workspace, which may sit below the Git root.

The canonical rule is now easy to state: **one typed folder per artifact type, with the slug on the file.** Pass 0 writes `cvg/docs/brd/<slug>.md` or `cvg/docs/no-go/<slug>.md`; Pass 1 writes `cvg/docs/tech-spec/<slug>.md`; Pass 2 uses `cvg/docs/adrs/`. Discovery tries those typed folders first, then the legacy flat `brd-*.md` and `tech-spec-*.md` shapes. Established bare directories remain compatibility fallbacks. An explicit path or `--tasks-dir` overrides discovery; Task-Spec's `TASKSPEC_BACKLOG_DIR` override remains available.

The conductor follows the same project-root and typed-folder decision. Its canonical prompts live at `cvg/brain/_prompts/`; its Pass 0 and 1 probes check `docs/brd/` and `docs/tech-spec/` first, then their legacy flat prefixes. Pointing the conductor at one workspace while a pass writes another would make the sequence board false.

This is not cosmetic. A valid workspace may live at `<repo>/projects/demo/cvg/`. If scripts anchor tasks, state indexes, profiles, or eval working directories to the repository root, they can read an empty backlog, create a shadow index, or execute against the wrong project. The current contract is uniform: resolve the tasks directory, derive the workspace from it, and run task evals there.

Tool root

`CVG_HOME` selects the installed Converge package and its scripts.

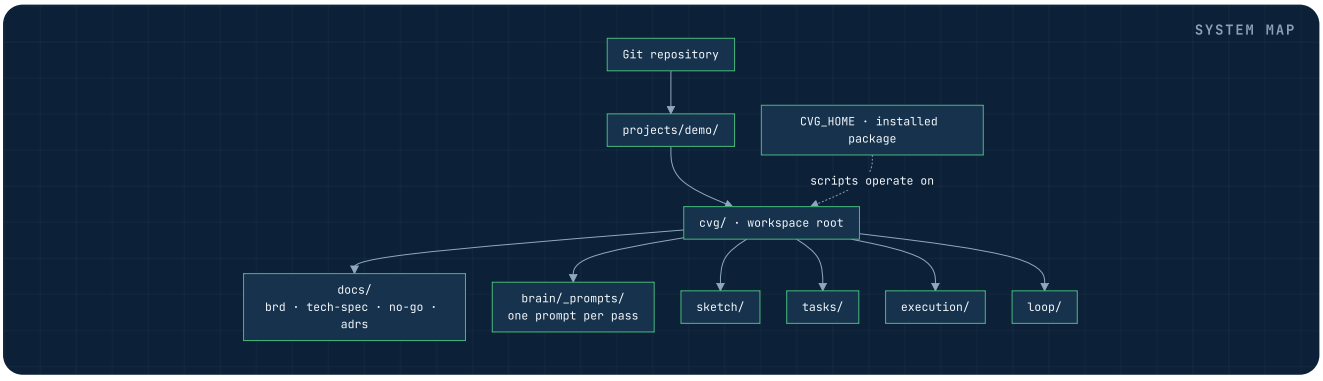
Project root

`CVG_PROJECT_ROOT` selects the consuming project's local `.cvg/` state.

The two roots are independent. This allows one installed toolchain to serve many projects without confusing package assets with project state. Tests must include a nested workspace because only that shape exposes an accidental Git-root anchor.

RESOLUTION PRECEDENCE	OPERATIONAL CONSEQUENCE
Explicit command path / `--tasks-dir`	Always wins; best for CI and audits.
`TASKSPEC_BACKLOG_DIR`	Overrides Task-Spec backlog discovery when explicitly set.
Canonical typed `cvg/*` **location**	Preferred project layout and shared workspace anchor.
Legacy flat prefix or bare directory	Additive compatibility fallback; an existing workspace keeps gating unchanged.

Design-command discovery treats zero or multiple candidates as safe failures and reports the search. Loop's suffix resolver is a known exception: it takes the first sorted match, so automation must supply an exact task ID/path or exact tracker reference. After task resolution, execution profiles, loop state, and eval working directories must all derive from the same workspace decision. A mismatch is a contract failure because it changes both inputs and possible side effects.



FOUNDATIONS · 10

A pass is a transformation contract

Every pass names inputs, a bounded method, durable outputs, a gate, and an honest failure route.

A numbered label alone does not make a process reliable. In Converge, every pass is closed by five explicit elements:

01 **Inputs:** the canonical upstream artifacts and any live system evidence the pass is allowed to inspect.

02 **Transformation:** a skill workflow that states the questions, altitude, and non-goals for the work.

03 **Outputs:** durable files that become the next pass's bounded input.

04 **Gate:** deterministic checks with one stable, greppable token and a documented exit-code contract.

05 **Failure route:** a named stop or return to the pass that owns the missing decision.

The conductor wraps, but does not alter, this contract. Before the transformation it checks prior artifact presence and supplies the pass prompt; after the transformation it checks that the output landed, then points to the same authoritative gate listed here. This is why `CONDUCTOR_POST=OK` and a failing pass token are not contradictory: one proves presence, the other evaluates the artifact.

Gates are deliberately narrower than sign-off. For example, `CHECK_ADR=OK` can establish that the ADR set satisfies structural and language rules; it cannot establish that the owner chose the right architecture. `CHECK_CONSENSUS=OK` proves the objection log is complete, closed, and fresh against the live plans; the owner must still approve the accepted risk. `CHECK_RUNTIME_CONTRACT=PASS` proves static profile freshness, hashes, topology, manifests, and guards; it does not attest the current host or prove future code is correct.

Default unattended path: fail closed, explain precisely. Missing input, stale hashes, unhonored capabilities, malformed evals, or ambiguous discovery must not be converted into a hopeful pass. The terminal token must say what class of outcome occurred so automation never parses narrative prose.

GATE CLASS	TYPICAL SUCCESS	TYPICAL RECOVERY
Structure/lint	Required sections and fields exist at the correct altitude	Repair the owning artifact; rerun without changing downstream work
Freshness/integrity	Live bytes match stamped hashes or HMAC	Review the change, then intentionally renew the stamp/profile
Parity/graph	Projection count and dependency edges match canonical specs	Rebuild the projection; never edit specs to fit the board
Static runtime profile	Hashes, topology, manifests, and guards are coherent	Repair or re-bind the profile
Current host	Separate doctor reports observed controls	Choose another host/control or stop for owner policy
Outcome	Eval, scope, integrity, and required holdout are satisfied	Iterate with evidence or land in a named non-success state

Command output and exit status are a pair. A stable token makes the semantic state greppable; the exit code controls orchestration. Draft, empty, usage, blocked, and exhausted outcomes must remain distinct even when several are non-zero. This is why wrappers preserve byte output and publish the taxonomy through `cvg agent-context`.

DESIGN · PASS 0

Capture — decide whether an idea deserves a build

Optional when a usable BRD exists; essential when the input is only a thought, pain, or request.

Pass 0, implemented by `idea-to-brd`, turns an unstructured idea into an owner-voice business requirements document—or a durable no-go. It asks frontier questions: every question whose prerequisites are settled appears in the same round, numbered, with a recommended default. Facts are looked up where possible; only decisions are returned to the owner.

STEER pass-0-capture.md CLOSE cvg capture

The BRD stays at business altitude. It records the affected actor, quantified pain, cost of inaction, target outcomes, KPIs in the owner's terms, explicit scope in and out, constraints, assumptions, and open questions with owners. It does not select frameworks, schemas, or file paths. The do-nothing test is first-class: if tolerable inaction beats the expected value of change, the correct output is a no-go record, not a forced project.

INPUT	Idea, request, pain report, or incomplete brief
OUTPUT	Owner-voice BRD under `docs/brd/`, or a durable record under `docs/no-go/`
GATE	`CHECK_BRD`
EXIT EVIDENCE	Provenance-tagged pain measure, owner KPIs, non-empty scope, owned questions

A client-supplied BRD can skip Capture and enter Intent directly, provided it is usable. Skipping an optional pass is not permission to carry unresolved business ambiguity into a technical spec. If Pass 1 cannot state the problem in one breath, return here.

```
cvg capture --draft cvg/docs/brd/observability.md
# CHECK_BRD=DRAFT_OK or CHECK_BRD=DRAFT_INCOMPLETE

cvg capture cvg/docs/brd/observability.md
# CHECK_BRD=PASS or CHECK_BRD=FAIL

cvg capture --no-go cvg/docs/no-go/observability.md
# CHECK_BRD=NOGO_OK or CHECK_BRD=NOGO_INVALID
```

The owner controls the canonical verdict and date. The checker strips fenced examples before looking for authorization, validates real ISO dates, rejects empty scope bullets, and requires every numbered pain measure to carry `(measured)`, `(estimated)`, or `(guessed)` on that line. Every guessed number must create an owned verification question. Owner voice and altitude remain human judgments; the gate warns rather than pretending grep can decide them.

When multiple stakeholders exist, name the tie-breaking decider. If a question survives two frontier rounds, preserve it with an owner instead of inventing an answer. A no-go record must include the decision owner, parking date, exact do-nothing rationale, and the condition that would reopen the idea. It is searchable portfolio memory, not deleted demand.

DESIGN · PASS 1

Intent — turn the brief into falsifiable requirements

The technical spec defines observable behavior, acceptance, and the decisions that materially change the build.

Pass 1, `brd-docs-to-tech-req`, reads the BRD as a senior engineer rather than transcribing it. It identifies the real problem beneath the requested solution, asks the few questions whose answers change the build, and crystallizes a technical requirements specification whose claims can be proven or refuted.

STEER pass-1-intent.md CLOSE cvg intent

Every requirement traces to a business outcome or constraint. Acceptance criteria use observable behavior and measurable thresholds. Scope is explicit. Non-functional requirements state their measurement method, not adjectives such as “fast” or “scalable.” Open decisions have owners; assumptions are marked as assumptions. The document is solution-aware only where a decision is genuinely required—it must not smuggle a full implementation plan into the intent layer.

Good requirement

Names the actor, trigger, observable response, boundary conditions, and a verification method tied to a BRD KPI.

Weak requirement

Names a preferred tool, promises “robustness,” or declares completion without a falsifiable observation.

INPUT	Usable BRD plus supporting source material
OUTPUT	Verifiable technical requirements specification
GATE	`CHECK_TECH_SPEC`
RETURN UPSTREAM WHEN	The outcome, owner, KPI, or material product decision is unresolved

Ambiguity killed here is cheaper than ambiguity discovered in a loop. Once the spec closes, Pass 2 may inspect the real system and decide which grounding choices deserve ADRs.

Gap register

Record each unknown as a typed record with severity, owner, and resolution. A blocker with a blank, “pending,” or otherwise sentinel resolution fails closed; only the named decision-maker may downgrade it.

Prior art

Research comparable solutions at the problem level to sharpen requirements. Do not import a vendor stack or physical schema merely because another implementation used it.

```
cvg intent --draft cvg/docs/tech-spec/observability.md
# CHECK_TECH_SPEC=DRAFT_OK | DRAFT_INCOMPLETE

cvg intent cvg/docs/tech-spec/observability.md
# CHECK_TECH_SPEC=PASS | FAIL | USAGE_ERROR
```

The BRD owner remains authority over outcomes; engineering owns the falsifiable translation and must show the trace. A PDF may be the signable delivery artifact, but the Markdown source is what the deterministic gate reads. Recovery from a failed gate is targeted: add the missing baseline, rewrite the wish as a threshold, resolve the blocker with its owner, or return to Capture when the underlying business question was never settled.

DESIGN · PASS 2

Structure — ground requirements against the real system

ADRs preserve hard-to-reverse decisions; CONTEXT.md preserves one shared vocabulary.

Pass 2, `tech-req-to-adrs`, reconciles the specification with the live terrain. In brownfield work it opens schemas, services, jobs, data flows, deployment boundaries, and existing contracts end to end. In greenfield work it names the assumptions the future system will stand on. It records only decisions that meet all three tests: difficult to reverse, consequential for downstream work, and plausibly decidable another way.

STEER pass-2-structure.md CLOSE cvg structure

Each numbered ADR states context, decision, alternatives, consequences, and status. The Decision section says what is true; it must not become a disguised implementation checklist. Consequences may use modal language because they describe what follows from the choice. Domain terms enter `CONTEXT.md` as they stabilize so later passes do not invent competing names for the same concept.

INPUT	Technical requirements plus the live system
OUTPUT	`cvg/docs/adrs/*` and `CONTEXT.md`
GATE	`CHECK_ADR`
STRONG EVIDENCE	Repository paths, schema definitions, interfaces, observed jobs, authoritative docs

Ground truth before plans. If a source is stale or the live system contradicts the spec, reconcile it now. Pass 3 may decompose accepted reality; it may not choose architecture that Pass 2 declined to record.

The resulting ADR set is intentionally small. Recording every reversible choice as an ADR obscures the decisions that actually constrain downstream work.

```
cvg structure --final --dir cvg/docs/adrs
# CHECK_ADR=OK | FAIL | EMPTY | USAGE_ERROR
```

The gate checks a directory as a set. `EMPTY` is distinct from a malformed set: it means no decision artifact was discovered and cannot authorize decomposition. Numbered filenames and explicit status make ordering and supersession visible. A stranger should be able to reconstruct why the decision existed, which alternatives were rejected, and what downstream work must now assume.

Live-system owner

Engineering is accountable for opening the executable path, not trusting diagrams or README claims that may have drifted. Evidence should cite concrete schema, interface, job, or source locations.

Recovery

If an ADR says “build X,” lift the plan out. If it contradicts reality, repair the ADR and its source evidence. If the choice is reversible and local, remove it from the ADR set rather than inflating governance.

Pass 2 and Pass 3 normally share the same reasoning session because the grounded understanding is richer than the files alone. If that session boundary is lost, reload and re-ground before decomposing; do not rely on memory to bridge it.

DESIGN · PASS 3

Decompose — cut along seams, then prove legs

A swimlane is a genuine system seam; a leg is an independently provable stretch inside that lane.

Pass 3, `reqs-to-swimlane-plans`, turns grounded requirements into a plan tree without falling to task or code altitude. It finds the joints already present in the system—feature boundaries, components, data ownership, protocols, or operational responsibility—and creates one focus-oriented plan per genuine seam under `cvg/sketch/swimlane-*/`.

STEER pass-3-decompose.md CLOSE cvg decompose

The decomposition chain is **seam → swimlane → leg → Task-Spec**. A lane owns one coherent responsibility. Each leg names a bounded change in that lane, its dependencies, the upstream ADRs it inherits, and one proving test. A steel thread crosses the minimum set of lanes needed to demonstrate end-to-end value early. The right number of lanes is the number of real seams, not a target imposed for symmetry.

Plan altitude

Responsibilities, boundaries, ordering, contracts, risks, proving tests, and integration points.

Too low

Atomic tickets, SQL statements, function signatures, line edits, or worker prompts. Those belong downstream.

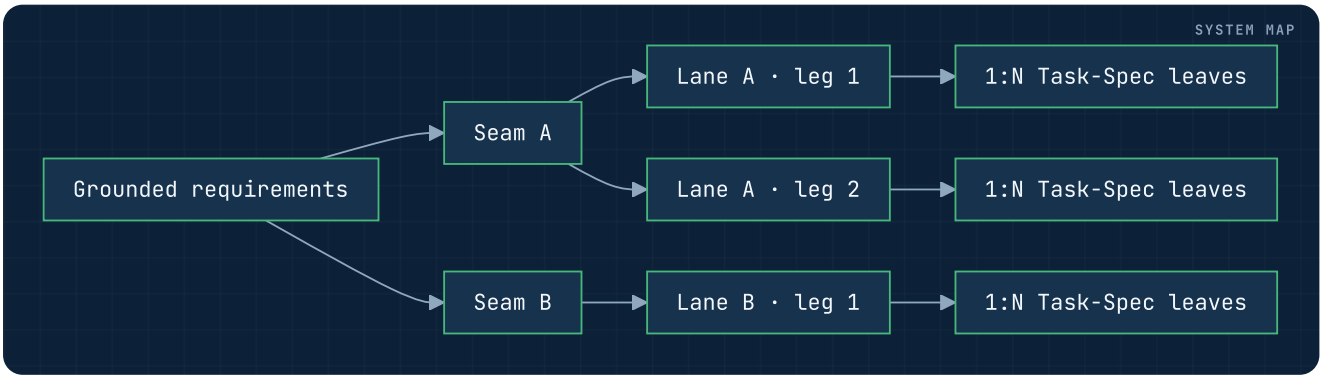
INPUT	Accepted requirements, ADRs, and shared vocabulary
OUTPUT	Swimlane plan tree with named legs and a steel thread
GATE	`CHECK_PLAN`
RETURN UPSTREAM WHEN	A seam depends on an unrecorded architectural decision

Pass 3 creates the surface that Pass 4 can attack. Plans that blend seams or hide dependencies make meaningful adversarial review impossible.

```
cvg decompose --dir cvg/sketch
# CHECK_PLAN=OK | FAIL | EMPTY | USAGE_ERROR
```

Each `swimlane-<seam>/` directory contains a lean `swimlane-<seam>.plan.md` index and contiguous leg files such as `swimlane-<seam>-leg-01-<tech>.md`. The stable leg key excludes the swappable technology label. The index links down; every leg links back to its parent. A leg carries one responsibility, declarative Given/When/Then proof seeds, independence, consumed/produced interfaces, appetite, and the 1:N Task-Specs it will yield—never executable eval code.

The machine gate checks lane metadata, exactly one `thread=yes` steel-thread lane, risk and owner fields, contiguous legs, bidirectional links, seam-evolution declarations, cycle handling, and altitude guards. Human review still decides whether the cut is genuinely deep, whether one lane must see another's internals, and whether a leg is independently finishable. A fuzzy interface is repaired by moving or folding the seam, not by adding prose around it.



DESIGN · PASS 4

Consensus — use a different family to try to refute

Agreement is earned by surviving attack, not produced by asking the author to self-review.

Pass 4, `sketch-plans-adversarial-review`, binds a different-family engine as a skeptic. The adversary begins from "refuted," attacks the plans one at a time, and looks for broken assumptions, missing dependencies, unsafe boundaries, unverifiable legs, contradictory ADRs, Goodhartable tests, and failure modes that the author normalized away.

STEER pass-4-consensus.md CLOSE cvg review --check

`cvg review --adversary <engine>` dispatches the attack through a read-only engine adapter. A comma-separated list can run multiple adversaries and merge their objections into one referee-stamped log. The referee holds no model credentials; each engine CLI authenticates itself. Input hashes and adversary provenance are recorded so the log cannot be reused after plans change.

REVIEW RESULT	MEANING
`REVIEW=OK`	A cross-family attack completed and produced a stamped result.
`REVIEW=SKIP`	No eligible engine ran; this is not consensus.
`REVIEW=TIMEOUT`	The attack did not finish inside the watchdog.
`REVIEW=ERROR`	Dispatch or merge could not establish trustworthy output.
`REVIEW=USAGE_ERROR`	The invocation contract was invalid.

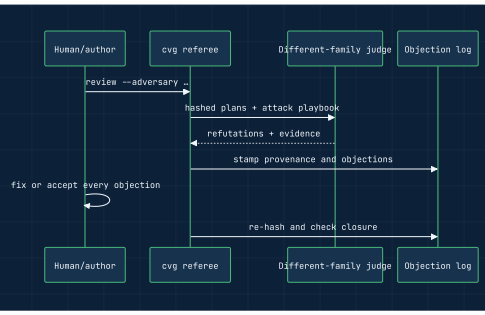
`cvg doctor` reports engine readiness and requires enough diversity to make cross-family review meaningful. The attack is not the gate: it creates objections. Closure happens only after every objection is resolved and the live plans still match their stamped provenance.

```
cvg doctor
# DOCTOR=OK | FAIL

cvg review --adversary codex,kimi
# REVIEW=OK | ERROR | SKIP | TIMEOUT | USAGE_ERROR

cvg review --check
# CHECK_CONSENSUS=OK | FAIL | EMPTY | USAGE_ERROR
```

The dispatcher sends the attack playbook and hashed plan set to a read-only engine process under a watchdog. Multi-adversary mode reuses the single-engine path, unions and renumbers objections, records `raised_by`, and recomputes provenance in one merged log. It fails closed if no cross-family engine actually ran; configuring two aliases that resolve to the same family does not manufacture diversity.

The reviewer also grounds its attack against the technical specification and ADRs. A plausible objection unsupported by those sources is still recorded, but its resolution must distinguish a real contract defect from an out-of-scope preference. When dispatch times out or skips, recovery is to repair engine readiness and rerun—the absence of objections is not evidence that none exist.

SYSTEM MAP



DESIGN · THE BARRIER

Close every objection, then ask the owner

The machine can prove the log is complete and fresh; only the owner can accept residual risk.

Every Pass 4 objection has one of two legitimate dispositions. **FIXED** means the plan changed and the response points to the repair. **ACCEPTED** means the risk remains intentionally, with a named owner and rationale. “Noted,” “later,” silent deletion, or an unowned exception is still open.

`cvg review --check` validates structure, resolution semantics, and provenance by re-hashing the live plans. It emits `CHECK_CONSENSUS=OK|FAIL|EMPTY|USAGE_ERROR`. An empty log does not prove consensus; it may mean no attack happened. A stale hash does not become acceptable because the changes look minor. Re-dispatch or re-resolve against the current plans.

The barrier has two locks.

1. The deterministic gate confirms no unresolved, malformed, or stale objection survives.
2. The owner explicitly signs off the hardened plans and any accepted risk.

Crossing the barrier changes the operating mode. Upstream artifacts remain human-led design; downstream work becomes machine-led build inside their constraints. The owner is not approving every future line of code. The owner is approving the problem framing, architectural ground, seams, proving strategy, and explicit residual risk from which atomic tasks may be derived.

If Pass 5 exposes a new seam or a task requires a product decision, the barrier was incomplete. Return to the owning pass, update artifacts, renew the attack, and sign off again. Do not patch semantic gaps inside a task.

OBJECTION FIELD	REQUIRED MEANING
Identity + source	Stable objection number, plan/leg reference, and adversary provenance.
Claim + evidence	The concrete failure or contradiction and the artifact evidence that supports it.
Disposition	`FIX` with the changed location, or `ACCEPT` with owner and reason.
Freshness	Input hashes recomputed against every current plan in the reviewed set.

Changing a plan during resolution is expected; closing the barrier on the old hash is not. The final check must observe the repaired bytes. A silent drop, unowned acceptance, missing reviewed file, empty log, or retired compatibility choice fails the machine half. An owner signature recorded before those checks cannot be carried forward as approval of the final plan set.

The handoff to Pass 5 should name the signed-off plan revision, unresolved non-blocking risks, and the leg identifiers that may now yield tasks. This makes the barrier a reproducible transfer of authority rather than a meeting memory.

BUILD · PASS 5

Tasking — make the definition of done travel

A Task-Spec is an atomic, vendor-neutral unit whose behavior, scope, budgets, and runnable proof live together.

Pass 5, `task-spec`, turns accepted plan legs into a dependency DAG of leaf tasks under `cvg/tasks/`. A runnable leaf declares goal, behavior, scope, dependencies, execution choices, budgets, evals, and an Exit Check. The implementer receives the same definition of done that the gates later use; completion criteria cannot be reconstructed from a chat transcript.

STEER pass-5-tasking.md CLOSE cvg tasks gate --stamp

Task-Spec supports three authoring profiles—`lite`, `standard`, and `full`—and six effort tiers. XS, S, M, and L are runnable leaves with increasing write-surface sizing guidelines. XL and XXL are decomposition nodes and must name children; they are never delegated directly. L requires a configured long-horizon backend, not one hard-coded vendor.

EFFORT	SHAPE	WRITE-SURFACE SIZING THRESHOLD
XS	Runnable leaf	1 path
S	Runnable leaf	2 paths
M	Runnable leaf	3 paths
L	Long-horizon runnable leaf	5 paths
XL / XXL	Decomposition node	Children required; never dispatched as a leaf

These path counts are not hard delegation maxima in 0.1.0. The validator emits a warning above the threshold and PRE may still pass; the author must split or reclassify deliberately. By contrast, the configured long-horizon backend requirement for an L leaf is enforced as an error.

INPUT	Accepted swimlane legs and their inherited decisions
OUTPUT	Task-Spec DAG with runnable evals and explicit dependencies
PRE GATE	`safe-to-delegate.sh --stamp`; Tier 1 for unattended execution
POST GATE	`accept-task.sh --stamp`; work, conditional Git scope check, and integrity

PRE seals delegation authority; POST proves the resulting work. They are duals, not aliases. A broken eval is an upstream defect: report and repair it, never weaken signed goalposts inside execution.

```
cvg tasks new export-format S
cvg tasks validate --no-state cvg/tasks/T-20260729-export-format.md
cvg tasks gate --stamp cvg/tasks/T-20260729-export-format.md
# TIER=1  (required for unattended execution)
```

The task author owns behavior and evals; the accepted-plan owner owns traceability; the worker owns implementation only. `cvg tasks validate` refreshes derived state by default; `--no-state` suppresses that metadata write. Plain `cvg tasks gate <spec>` does not write Task-Spec or Converge metadata, while `cvg tasks gate --stamp <spec>` first removes any old signature, then writes a fresh sign-off envelope only after checks pass. A failed re-seal can therefore leave structurally signed frontmatter without its former cryptographic seal. Inspect the resulting tier before dispatch. Every task command must resolve the same backlog root.

Runnable evals must fail before the promised behavior, inspect the public contract, and emit diagnostic evidence. Destructive or network-dependent setup needs explicit fixtures or blocks delegation. The Exit Check composes the terminal claim; it is not another prose criterion.

BUILD · TASK CONTRACT

Inside the cornerstone unit

Six zones separate intent, behavior, proof, constraints, operations, and recovery.

A Task-Spec combines YAML frontmatter with six human-readable zones. The frontmatter carries machine-resolved identity and control fields: task ID, status, profile, effort, dependencies, write and create paths, backend and agent choices, budgets, sign-off envelope, acceptance state, and optional tracker reference. The body explains the work without duplicating those controls in prose.

ZONE	PURPOSE
1 · Intent	Goal, rationale, inherited decisions, and explicit non-goals.
2 · Behavior	Numbered observable behaviors and boundary cases.
3 · Contract	Success criteria, runnable evals, validation card, and Exit Check.
4 · Guardrails	Allowed paths, forbidden actions, dependency and safety limits.
5 · Operations	Open Questions, honest unknowns, and blocker conditions.
6 · Rollback + observability	Full-profile recovery, signals, and failure visibility.

For standard and full profiles, structural validation and PRE hard-fail a behavior with no proving eval and a `verifies` reference to an undeclared behavior. They warn when an eval lacks reverse mapping. `cvg tasks dod` hard-fails orphan behaviors and evals without `verifies`, but it does not reject dangling IDs. Full bidirectional coverage therefore requires validation/PRE *plus* DoD. The Exit Check combines the evals into the terminal task verdict.

The spec remains canonical even when projected to a tracker. A tracker issue may summarize goal, status, and dependencies, but it must link back to the sealed file rather than becoming a mutable second instruction source.

Profiles

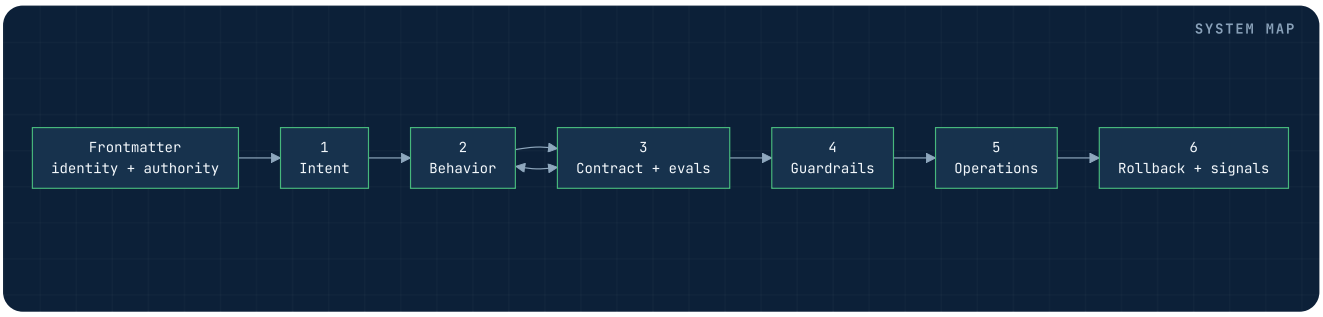
`lite` keeps a small, reversible leaf concise; `standard` is the default and requires full behavior/eval traceability; `full` adds the recovery and observability detail expected for material blast radius. The `profile` frontmatter field is not HMAC-covered, though changing it after Bind changes the whole-file hash and invalidates the epoch.

Lifecycle

`ready → in-progress → blocked|done|parked` moves through the locked transition command. For an actual state change into `done`, the tool requires stamped `accepted: true` plus `cvg/receipts/<id>.json` with the same `task_id` and `result: pass`. A same-state `done → done` request returns NOOP before re-running that guard.

The PRE HMAC covers body bytes and authority-bearing frontmatter: declared touch/create paths, dependencies, effort, execution backend, agent choice, budgets, and requirements. Changing any of them creates a new delegation decision. Field names are extracted in fixed order, but their YAML blocks and the extracted body are syntax-sensitive; equivalent indentation, block/inline style, list formatting, or prose reflow can change the digest. POST attempts keyed HMAC verification when the key, signature, and crypto provider are available; otherwise it reports structural-only integrity and may still accept. Policy that requires Tier 1 must enforce that requirement separately. POST also runs evals in the current workspace and, when Git data is available and `--no-blast-radius` is absent, checks scope; otherwise acceptance can proceed with scope unchecked. It may reconstruct the unpatched baseline in a temporary worktree for optional gold-sanity. Baseline reconstruction is a discrimination check; it is not where candidate work is accepted.

`cvg tasks dod <spec>` renders the definition of done and returns `DOD=COMPLETE|GAPS`. Use it before signing; an untraced behavior is an authoring defect, not something the worker should infer.



BUILD · PASS 6

Register — project the sealed backlog, never replace it

The tracker is an opt-in operational shadow of canonical Task-Spec files.

Pass 6, `task-specs-to-issues`, maps one signed-off Task-Spec to one tracker issue and every `depends_on` edge to one `blocked-by` relationship. It supports GitHub, Linear, Jira, and a fake adapter for offline tests. Registration is idempotent: the issue-side marker is the identity key, so reruns update rather than duplicate.

STEER pass-6-register.md CLOSE cvg register --check

The projection may stamp a `tracker_ref` receipt back into the spec, while the issue links to a branch-pinned source URL. The referee does not hold tracker credentials; the selected adapter authenticates itself. A six-verb adapter boundary keeps tracker-specific APIs outside the canonical method.

COMMAND	CONTRACT
`cvg register`	Project sealed specs and edges; public dry-run emits `DRY_RUN=OK`; normal run emits `REGISTER=OK|FAIL|EMPTY`.
`cvg register --check`	Read-only 1:1 parity, edge, cycle, and leak check; emit `CHECK_REGISTER=OK|FAIL`.
`cvg setup tracker …`	Validate and record adapter choices, never credentials.

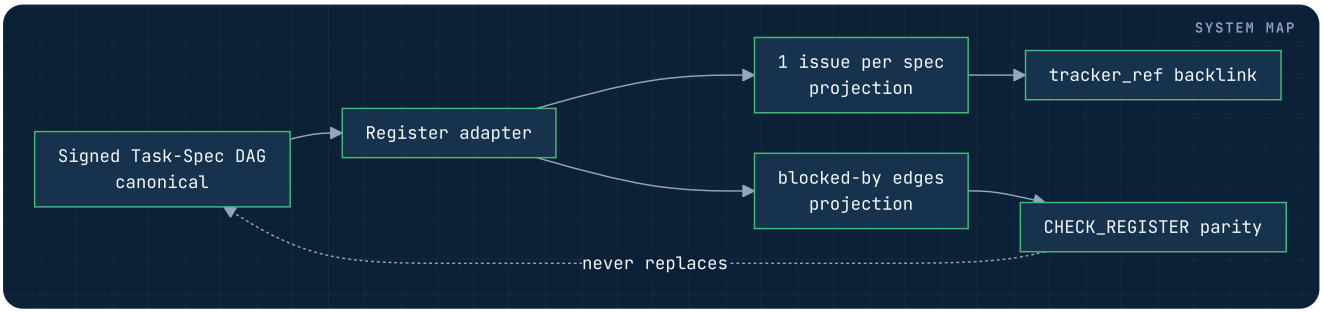
Pass 6 is optional because a repo-local ready frontier is sufficient for single-task operation. If used, the board remains downstream of Git truth. Tracker write failure is accounted for but does not retroactively redefine a task's eval verdict. State transitions must remain evidence-led: an issue is not closed merely because an agent claims completion.

```
cvg register --tracker linear --dry-run
# DRY_RUN=OK  (public wrapper)
cvg register --tracker linear
# REGISTER=OK | FAIL | EMPTY
cvg register --check
# CHECK_REGISTER=OK | FAIL
```

Direct `register.sh --dry-run` retains its native `REGISTER=DRY_RUN` token; the public `cvg` wrapper intercepts the invocation first.

Register filters active specs by `signed_off: true` and `registerable: true` only; it does not validate HMAC/tier or leaf effort. Normal PRE prevents legitimate node stamps, but hand-stamped or malformed specs can be projected unless validation or gating runs upstream. The adapter upserts by the issue-side Task-Spec marker, then creates each dependency edge after both endpoints exist. The parity check compares counts, IDs, backlinks, edge direction, cycles, and ungated leaks. It is read-only. A mismatch is recovered by replaying projection from canonical files—not by changing `depends_on` to match a hand-edited board.

Optional structure projection can create Initiative → Project → Milestones, but it is disabled by default and cached in private, symlink-resistant local state. Identity mapping is create-only and fails soft when absent; it must not change issue identity or the task contract. The adapter holds its own API credential, while `.cvg/config` records only backend, team, and projection choices.



BUILD · PASS 7

Bind — connect one signed task to an honest runtime

Pass 7A emits the enforceable contract; Pass 7B emits the minimal brief the worker reads.

Pass 7, `task-to-runtime-contract`, begins by resolving and verifying one runnable Task-Spec. It binds the exact spec revision to the smallest evidence slice needed for that task, chooses an initial task-local topology, generates portable path guards and runtime adapter manifests, and gates the result. The profile is stored beneath `cvg/execution/<task-id>/`.

STEER pass-7-bind.md CLOSE cvg bind --check

7A is derived runtime control. The `execution-profile.yaml` names the epoch, spec hash, evidence hashes, topology, writable and creatable paths, required capabilities, external-write policy, a hash-bound pointer to canonical Task-Spec budgets, and the mechanism by which an adapter claims to prevent, detect, or cannot honor a constraint. The profile is source-hash-bound control-plane state, not independently signed authority.

7B is model context. `AGENTS.task.md` carries IDs, Goal, exact allowed and forbidden paths, the actual Exit Check, network and external-write boundaries, assurance, closure instructions, and a pointer to the project router. It is compact but not identifier-only, and it does not synthesize broad doctrine. The worker must read canonical, hash-bound sources from disk.

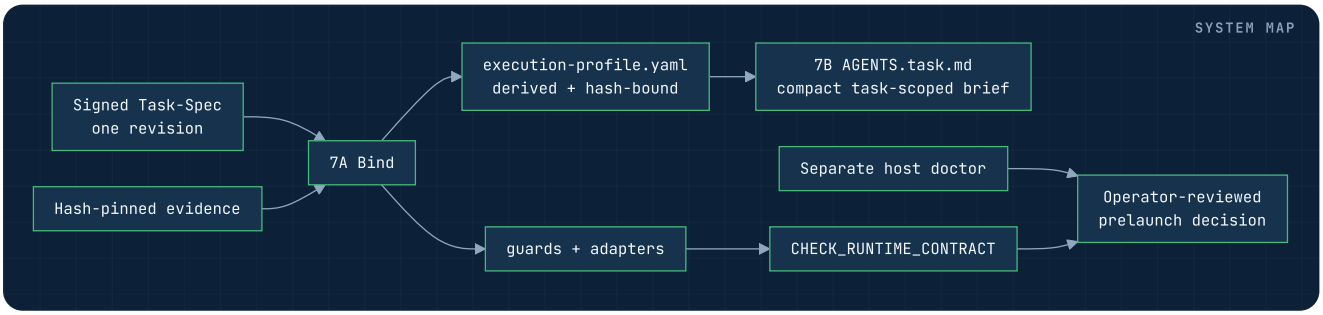
DEFAULT INPUT	Tier-1 signed leaf. `--supervised` can bind Tier 2, but does not prove a supervisor is present at Loop launch.
OUTPUT	Execution profile, guards, adapter manifests, task brief
STATIC GATE	`CHECK_RUNTIME_CONTRACT=PASS|FAIL` for profile, source hashes, topology, manifests, and guards
READ-ONLY RECHECK	`cvg bind --check --task <spec>`

Bind owns neither task meaning nor execution. Pass 5 owns behavior, routing, and budgets; Pass 8 owns attempts and settlement. Bind derives capability, provider, and settlement policy. Credentials remain host-owned. A separate `cvg doctor runtime-contract` probe reports current-host readiness; Bind and Loop do not invoke or integrate that attestation automatically.

```
cvg doctor runtime-contract --runtime generic
# DOCTOR_RUNTIME_CONTRACT=OK | DEGRADED | FAIL

cvg bind --task cvg/tasks/T-20260729-export-format.md
# CHECK_RUNTIME_CONTRACT=PASS | FAIL

cvg bind --check --task cvg/tasks/T-20260729-export-format.md
# run at project root, or pass --profile explicitly
# read-only static freshness + profile recheck
```

The binder defaults to one worker and requires static justification for any task-local topology. Evidence references are resolved and hashed; portable path guards are compiled from touch/create scope; runtime adapters declare concrete control mechanisms. The check recomputes Task-Spec sign-off directly rather than invoking a delegation gate that would execute project evals—running untrusted project code is not a read-only freshness check.

Failure recovery is explicit. A stale spec or evidence hash requires re-binding. A missing source returns to the evidence owner. A static capability gap returns to runtime choice or visible waiver policy; a current-host doctor failure requires operator repair before launch. A broad or empty path set returns to Tasking. Pass 8 must recheck Bind immediately before writing, because an earlier PASS says nothing about a later revision.



BUILD · RUNTIME AUTHORITY

The capability envelope declares closure

The profile names intended closure events; release 0.1.0 still needs the operator to close or replace authority.

Runtime authority is expressed as an epoch: `<task-id>@<spec-sha12>`. Capabilities are scoped to that epoch and the Task-Spec's own paths. The profile declares `settle`, `block`, `budget_exhausted`, and `epoch_change` as intended closure events. Release 0.1.0 does not persist or enforce a revoked state; cancelled, stalled, and error are absent, and a terminal token does not invalidate the profile. Operators must close authority and re-bind before reuse.

Each capability is accounted for with an assurance level:

ASSURANCE	MEANING	EXAMPLE CLASS
Prevent	The runtime stops the violation before it occurs.	OS sandbox, pre-tool hook, worktree boundary
Detect	The runtime may allow the action, but deterministic postflight can refuse settlement on Git-visible repository changes.	Portable diff/path guard
Cannot honor	The selected host has no truthful mechanism for the required control.	Unavailable isolation or network control

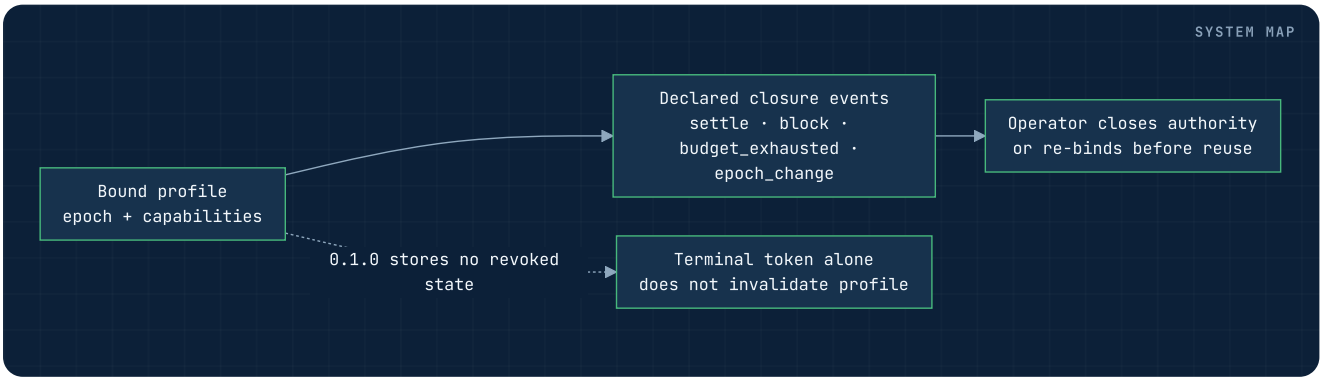
`cvg doctor runtime-contract` separately probes what the current host can genuinely enforce. It does not feed a result into Bind or Loop automatically; prelaunch policy must require and reconcile that attestation with the static profile. Silence is not a waiver. Marketing language such as "sandboxed" is not evidence unless mapped to a concrete host control.

In default bound mode, write scope is least privilege. Settlement stages only paths granted by `fs.write`; it never uses broad `git add -A`. Candidate and staged Git-visible paths are checked again. Ignored local bytes and out-of-repository effects remain outside that portable evidence and require an installed pre-tool hook, sandbox, or OS control to prevent. `--legacy-no-contract` instead uses repository-gate-only tracked-change staging; it is not task-scoped.

```
cvg bind --task cvg/tasks/T-20260729-export-format.md --runtime codex   --require net.egress
# static PASS only when the adapter manifest declares the required mechanism

cvg bind --task cvg/tasks/T-20260729-export-format.md --runtime generic   --require net.egress --accept-unenforced
net.egress
# explicit WAIVED record; assurance becomes unenforced
```

In a bound profile, `fs.write` is required and derives exactly from `touches_paths + creates_paths` minus every Do-Not-Touch fence. Network egress is denied unless the task's requirements justify it. The profile's headline assurance is the weakest required capability, so one unenforced grant cannot hide behind several prevented controls. Re-binding is cheap and mandatory after any spec byte changes because the old epoch no longer names the authorized revision.

The separate host probe reports observed primitives—runtime binary, isolation, worktree behavior, hooks—not vendor promises. Worktree availability is reported but does not itself determine the doctor verdict. An unenforced waiver is visible only in the unsigned derived profile: it has no owner, timestamp, or signature and is not copied into the 0.1.0 receipt. Record owner approval separately. Credentials stay at the harness/host boundary.



BUILD · PASS 8

The Loop — one assigned task, driven to evidence

Pass 8 knows one task deeply; it never decides which task the fleet should run.

Pass 8, `task-loop`, takes one issue identifier supplied by a human or CI. On the default unattended path it resolves a signed Task-Spec and current Pass 7 profile, rechecks `CHECK_RUNTIME_CONTRACT=PASS`, loads bound evidence, and operates inside that contract. The kernel then performs a bounded refinement cycle: attempt, run the task's own eval, feed exact failure evidence into a fresh attempt, and repeat while pre-attempt budget and progress checks permit.

`STEER` pass-8-loop.md `CLOSE` cvg tasks accept after settlement evidence

`--legacy-no-contract` is a visible supervised migration waiver: it bypasses Bind, warns that execution is unsigned, uses weaker tracked-change staging, and writes no execution receipt. It is not equivalent to the normal unattended evidence chain.

On GREEN, the settlement leg rechecks path policy, optionally invokes a Tier-2 judge whose available family-independence strength is recorded, stages only granted paths, commits, and opens a PR only when external writes are allowed. On RED, it either continues with evidence or lands in a named non-success state with a handoff or blocked report.

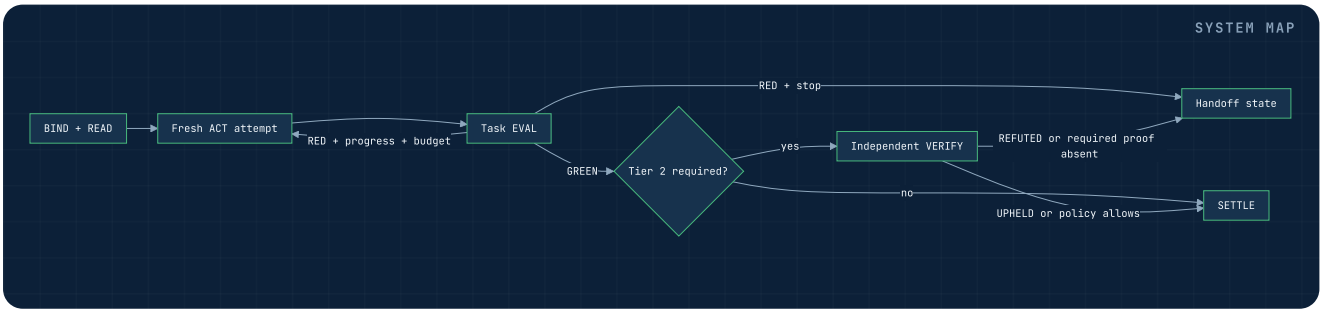
STEP	INVARIANT
Bind + read	Default path: current profile, valid sign-off, present evidence, one task; legacy waiver is weaker and explicit.
Act	Default bound mode: fresh engine, scoped paths, signed source. Legacy mode may execute unsigned, non-task-scoped work.
Eval	The Task-Spec's own runnable Exit Check decides level-1 GREEN.
Verify	Optional Tier 2 may be policy-selected; family independence is recorded, not universally required.
Settle	Recheck scope, obey external-write policy, attempt the pass receipt last.

`cvg loop --issue <id>` is shipped. Fleet selection, fan-out, and concurrency coordination are not hidden inside it; those are future Manager responsibilities.

```
cvg loop --issue T-20260729-export-format --agent codex   --max-iterations 3 --max-seconds 1800 --max-tokens 80000

cvg loop --issue T-20260729-export-format --estimate
cvg loop --issue T-20260729-export-format --gate-only
cvg loop --issue T-20260729-export-format --resume
# full non-dry kernel landing: TASK_LOOP=<state>
# estimate: ESTIMATE=CEILING
# normal text dry-run: DRY_RUN=OK
# public gate-only RED/parser failure: non-zero and may emit no TASK_LOOP
```

Every mode still requires an exact `--issue`; no Loop invocation selects its own work. The signed spec, current profile, and durable loop state resolve under the same workspace. `--estimate` reports configured ceilings without forecasting currency. Normal `--dry-run` stops before worker and settlement effects; `--gate-only` is the explicit single-shot eval and settlement path, so combine its dry-run flag with the documented effect contract.

Dispatchers must choose from the ready frontier before Loop. Command-line limits may tighten, never raise, the resolved budget. Release 0.1.0 still has a duplicate-name parsing gap for two body-sealed brake fields; the whole-file Bind hash catches later edits but is not a substitute for canonical field placement.



BUILD · LOOP MECHANICS

Fresh context, an iteration cap, and pre-attempt brakes

Bounded retry behavior is implemented in the runtime, with an explicit enforcement boundary.

Each attempt runs as a fresh engine process briefed from disk. Persistent evidence is split across Task-Specs, profiles, `state.env`, attempt logs, `HANDOFF.md`, and settlement receipts—not an ever-growing conversation. This prevents retry context from expanding quadratically and lets the next attempt read canonical evidence plus the most recent failure.

Before another worker attempt, the kernel checks the iteration cap, accumulated working time, and—when `budget_tokens` or `--max-tokens` resolves to a positive value—worker usage reported as `ENGINE_TOKENS`. Missing usage remains unknown. Command-line flags may tighten resolved limits but cannot raise them. Loop checks `<active-worktree>/cvg/loop/<task-id>/STOP` at the next attempt boundary. Under default isolation, a STOP created in the original checkout is not observed; use the printed temporary-worktree path. The signal cannot interrupt an in-flight worker or Tier-2 judge. When observed, it is removed and Loop lands `CANCELLED`.

Release 0.1.0 does not interrupt an in-flight worker attempt or recheck the run-level time/token counters before Tier 2 or settlement. One attempt may cross a threshold and, if its eval is green, continue onward. Tier-2 usage is not debited. Wall-clock and token values are therefore pre-attempt brakes, not strict end-to-end ceilings.

A stagnation detector fingerprints failure. If the same check fails in the same way for the configured no-progress count, the loop lands `STALLED` rather than burning the remaining budget. When a pre-attempt limit refuses another launch despite progress, it lands `EXHAUSTED`, preserves work-in-progress, and writes `HANDOFF.md`. Resume is explicit; a planned landing is not mislabeled success.

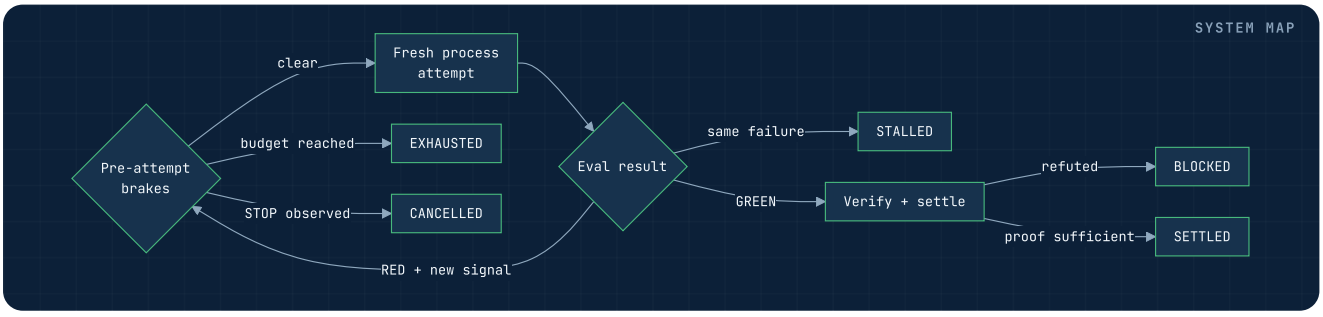
Current boundary. Resume rechecks the current binding during eval preflight, but shell-sources an unauthenticated `state.env` that is not tied to the prior binding, base, workspace, checkpoint, or receipt. In-place resume can reconnect retained state; default fresh-worktree resolution may restart at attempt 1 instead of restoring the prior tree.

Engine adapters expose only availability and prompt-file invocation. The kernel remains vendor-neutral and wraps hung CLIs with a watchdog.

The machine checkpoint is `state.env`. It stores `ITER`, `STRIKES`, `TOKENS_USED`, `STARTED_AT`, `ELAPSED_PRIOR`, and `LAST_FINGERPRINT`. In 0.1.0, `--resume` shell-sources those values without validating a checkpoint schema, task/binding/base identity, or staleness. Attempt logs and `HANDOFF.md` carry the last eval evidence and recovery instructions, but the resume loader does not validate them. The fingerprint must still distinguish meaningful new signal from an identical failure.

BRAKE	CHECKED BEFORE	LANDING
Iteration cap	Starting another worker process	`EXHAUSTED` + handoff
Accumulated-time brake	Starting another worker process; not before Tier 2	`EXHAUSTED`
Optional reported-token brake	Starting another worker process, using worker-adapter reports only	`EXHAUSTED`
No-progress count	Repeating the same failing hypothesis	`STALLED`
STOP file	Top of the worker-attempt loop only	`CANCELLED`

Recovery begins from `HANDOFF.md`, not a remembered chat. The owner decides whether to repair an upstream artifact, narrow the task, or authorize a new signed budget. Resume never implies that a worker may ignore the original ceilings.



BUILD · SETTLEMENT

Eight terminal states make failure legible

Only settled, locally settled, and no-op outcomes exit zero.

TOKEN	MEANING	SUCCESS?
`TASK_LOOP=SETTLED`	In a non-dry run: green eval, required verification satisfied, external writes allowed, branch pushed; PR opened when `gh` exists, otherwise a manual PR body is printed.	Yes
`TASK_LOOP=LOCAL_SETTLED`	In a non-dry run: green and committed locally; external writes were not enabled.	Yes
`TASK_LOOP=NO_OP`	Level-1 eval was green before an attempt; no path policy, Tier 2, settlement, or receipt ran. Closure still needs an existing matching receipt.	Yes
`TASK_LOOP=BLOCKED`	Kernel `--no-agent` RED, Tier-2 refusal, or settlement refusal. Public `--gate-only` RED may exit 1 with a blocked report and no token.	No
`TASK_LOOP=STALLED`	The same failure repeated without progress.	No
`TASK_LOOP=EXHAUSTED`	A pre-attempt iteration, elapsed-work, or configured reported-token check refused another worker launch; handoff written.	No
`TASK_LOOP=CANCELLED`	A STOP file was honored at the next worker-attempt checkpoint.	No
`TASK_LOOP=ERROR`	Task/contract resolution or another safe-continuation invariant failed.	No

Successful settlement is ordered. The Git-visible candidate diff is checked against Task-Spec scope, staging uses only granted paths, and the staged set is checked again. Commit precedes external publication; a pass receipt is attempted last for `SETTLED` or `LOCAL_SETTLED`. Other terminal states do not imply a new pass receipt.

Generated 0.1.0 profiles deny external writes, so a green run normally stops at a local commit and reports `LOCAL_SETTLED`. Push and PR require `--allow-external-writes`; Task-Spec has no signed publication-allow field. Tracker updates are accounted with a separate token; a board bridge failure is visible but must not dictate the code verdict.

```
# Examples of paired terminal evidence
TASK_LOOP=LOCAL_SETTLED   # exit 0
TRACKER=SKIPPED

TASK_LOOP=BLOCKED         # non-zero
TRACKER=FAILED
```

In non-dry runs, the first three Loop states exit zero: externally pushed with either an opened PR or manual PR handoff, locally committed under policy, or level-1 green before a worker attempt. Every other state exits non-zero even when its handoff is excellent. The token names current controller behavior; downstream automation must inspect the printed PR outcome rather than infer that `SETTLED` always means a PR object exists.

Two text-mode dry-run shapes. Normal Loop exits early with `DRY_RUN=OK` and no `TASK_LOOP`. Gate-only dry-run emits `TASK_LOOP=LOCAL_SETTLED|SETTLED` only when eval and path policy pass. Eval RED or path refusal exits 1 with a blocked report and no `TASK_LOOP`. No dry-run branch commits, pushes, opens a PR, or writes a receipt. Always interpret the mode together with the token.

Current receipt gap. Release 0.1.0 attempts the result receipt last, but receipt-write failure does not by itself revoke a `SETTLED` or `LOCAL_SETTLED` token. Treat that combination as incomplete evidence. The separate `transition … done` guard refuses completion unless a parseable matching `result: pass` receipt actually exists.

If publication or another outward leg fails, recover from the scoped commit and recorded error rather than rerunning implementation blindly.

BUILD · INDEPENDENT PROOF

Tier 2 adds an adversarial second reader

The Holdout is omitted from the brief but remains readable in the canonical spec; this is review diversity, not enforced test secrecy.

`cvg verify --task <spec> --judge <engine>` asks a selected model to grade the diff against the Task-Spec's intent and a holdout omitted from `AGENTS.task.md`. Loop still tells the worker to open the canonical Task-Spec, where that section is readable; 0.1.0 provides adversarial second review, not enforced train/test secrecy. Auto-selection compares judge family only with a recognized Task-Spec `execution_backend`—not the actual Loop `--agent`. An explicitly requested or sole installed engine may be same-family. Default text prints the judge and strength; JSON records `independence: independent|same-family|none`. Loop scans the verdict token and does not enforce the recorded strength. "Independent" is meaningful only for a recognized backend; a missing or unrecognized backend currently labels any judge independent.

VERDICT	RUNTIME MEANING
`CHECK_VERIFY=UPHELD`	The selected judge's attack found the work consistent with intent and holdout.
`CHECK_VERIFY=REFUTED`	A concrete violation was found; settlement is refused.
`CHECK_VERIFY=UNAVAILABLE`	No installed judge engine established a verdict.
`CHECK_VERIFY=ERROR`	The verification process failed safely.

Risk policy decides whether unavailable verification is tolerable. With no judge, low-blast direct verification can return `UNAVAILABLE` /exit 0; high blast returns `ERROR` unless direct `cvg verify --accept-unverified` is explicitly used. Loop has no passthrough for that waiver. No verdict is silently converted to an uphold.

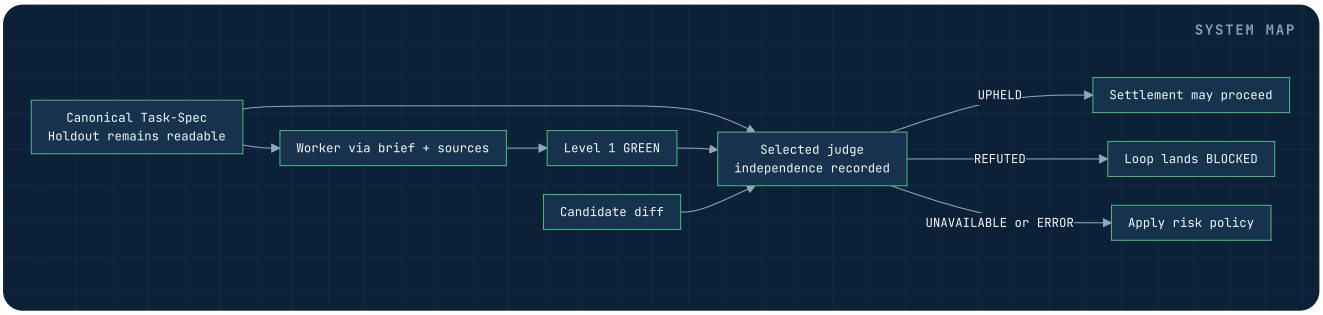
Missing verifier gap. If the verifier file itself is missing, Loop currently prints that Tier 2 is skipped and continues to settlement. Prelaunch policy must verify the file exists when second review is required.

Release 0.1.0 demonstrated both outcomes: one cross-family holdout upheld work; another refuted a green eval whose error path reported false health. The kernel refused settlement and wrote a handoff—proof that independent review can change the terminal state.

```
cvg verify --task cvg/tasks/T-20260729-export-format.md --judge kimi
# CHECK_VERIFY=UPHELD | REFUTED | UNAVAILABLE | ERROR
```

`## Holdout` is excluded from the 7B brief, not hidden from the repository. The judge sees the candidate diff, intent, and Holdout. It is grading-only by prompt, but the external judge CLI runs with ambient host permissions; 0.1.0 does not enforce a read-only sandbox. Direct `cvg verify` does not independently validate PRE or its HMAC; establish that integrity upstream. Inspect the recorded independence value; callers that require a different family must reject `same-family` themselves and select another installed engine explicitly.

The prompt requests confidence, findings, and reasoning, but the shipped parser accepts a JSON object containing only `{"verdict":"REFUTED"}`; bare text `REFUTED` is not parseable. Detail completeness is not machine-enforced. In Loop, refutation lands `BLOCKED` and writes a handoff; any repair, re-binding, or later `--resume` is a separate operator-authorized action. The verifier defaults to 300 seconds and Loop exposes no direct timeout flag; configure `CVG_VERIFIER` when needed. A judge that never grades the diff yields `UNAVAILABLE` / `ERROR`, never an uphold.



OPERATING SYSTEM · TRUST

The trust model: sign intent, bind authority, verify work

Converge separates instruction integrity, runtime enforcement, and outcome evidence.

Three controls answer three different questions. The Task-Spec PRE gate asks whether the delegation packet is structurally runnable and unchanged. Pass 7 statically binds the exact revision to declared capabilities, manifests, hashes, and guards; a separate host doctor probes current-host readiness. Loop decides settlement and attempts its execution receipt; the separate POST referee later decides whether Task-Spec acceptance may be stamped.

01 **Seal.** Tier-1 HMAC covers the task body and authorization-bearing fields, including touch/create paths, dependencies, effort, backend and agent choices, budgets, and requirements. The extracted YAML blocks and body text are syntax-sensitive.

02 **Bind.** Evidence hashes, epoch, path guards, runtime assurance, topology, and external-write policy become an execution profile.

03 **Execute.** The worker reads canonical sources. Installed controls can prevent; portable postflight can only detect Git-visible repository-scope violations before settlement.

04 **Prove.** Runnable evals, Exit Check, path checks, optional gold sanity, and tier-2 holdout establish outcome evidence.

05 **Close and receipt.** A terminal state signals intended closure but does not invalidate the stored profile. Close authority operationally, confirm the best-effort receipt, and re-bind before reuse.

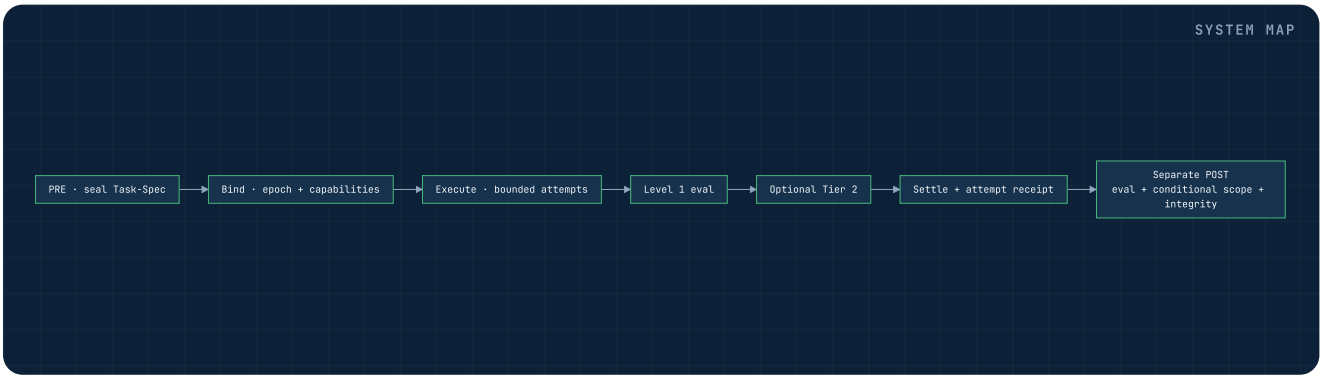
Task-Spec's trust ladder distinguishes valid keyed HMAC, supervised or structural legacy evidence, and malformed or mismatched input. PRE may pass without keyed proof only at Tier 2, for supervised dispatch; only Tier 1 is appropriate for unattended delegation. A signature does not make content wise; it prevents silent mutation after owner-approved intent and authorization were sealed.

The referee itself minimizes secrets. Model and tracker CLIs authenticate at their adapter boundary. Local signing keys and identity mappings are symlink-resistant, private, and never copied into project documentation.

SIGN-OFF TIER	EVIDENCE	DISPATCH POLICY
Tier 1	Key present, signature present, MAC verifies	Unattended delegation permitted
Tier 2	No key or legacy structural envelope	Loud warning; supervised dispatch only
Tier 3	Key present with mismatch or malformed signature	Hard fail; treat as tampered and intentionally re-stamp

Executor conformance is a separate axis. L0 reads the format and runs evals. L1 additionally acquires the lifecycle lock and closes state correctly. L2 additionally honors retry budgets and parks an unsatisfied task instead of looping forever. `conformance-check.sh --level L2 --executor "<cmd>"` makes adapter behavior testable and emits `CONFORMANCE=L2` on success.

A Tier-1 task can still be handed to a non-conformant executor, and an L2 executor can still receive a tampered spec. Trust requires both axes plus the Pass 7 runtime profile and post-execution evidence.



OPERATING SYSTEM · CLI

The CLI is referee, router, and conductor entrypoint

cvg exposes one coherent surface while preserving the behavior of proven pass scripts.

The core CLI consists of `bin/cvg` and its UI helper. Commands resolve the workspace, validate invocation, and route to the scripts that already implement each pass. The governing rule is “wrap, do not rewrite.” Wrapped output remains byte-identical; presentation belongs to help, version, and error surfaces, not around gate tokens.

The CLI is dependency-light: Bash 3.2 compatible, ShellCheck-clean, and free of mandatory pip or npm dependencies on the core path. Color degrades to plain text when output is piped, `NO_COLOR` is set, or `CVG_COLOR=0`. Every state remains understandable without ANSI.

RESPONSIBILITY	CLI BEHAVIOR
Discovery	Resolve explicit inputs or one unambiguous canonical workspace artifact.
Sequencing	Expose the conductor's read-only evidence board, prompt path, and next gate through `cvg next`.
Routing	Invoke the pass’s proven script without changing its semantics.
Contracts	Preserve stable tokens, exit codes, dry-run behavior, and JSON envelope.
Adapters	Keep vendor and tracker specifics behind narrow interfaces.
Self-description	Publish the command tree and machine protocol through `agent-context`.

Design-command discovery refuses ambiguity and reports candidates. Loop’s suffix resolver is weaker: it currently takes the first match. Automation must use an exact task ID/path or exact tracker reference; selecting the wrong task is a semantic mutation, not a convenience.

Package resolution

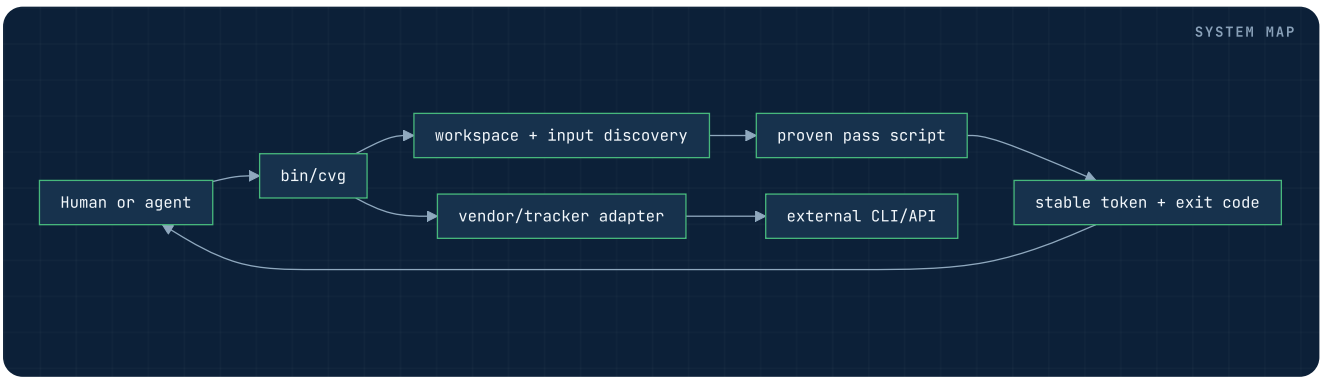
`CVG_HOME` identifies the installed scripts. Version and manifests come from that same package root so copied skills cannot silently report a different release.

Project resolution

`CVG_PROJECT_ROOT` and explicit workspace inputs identify project state. The CLI exports one resolved backlog decision to every Task-Spec subcommand.

Most usage errors pair a machine token with exit 2, but Loop unknown-option/missing-value parser exits can occur before `TASK_LOOP` is emitted. Always combine output with exit code. Wrapped gate failure passes through unmasked. Mutating commands support dry-run where declared. A verdict mode that avoids Task-Spec/Converge metadata writes can still execute project evals and their side effects; “no stamp” is not a general side-effect-free guarantee. S-or-larger additions to the CLI are themselves authored and accepted as Task-Specs, making the tool dogfood its autonomy contract.

Core commands remain Bash 3.2 compatible and parse-stable when piped. A UI improvement may change interactive color or spacing but must not decorate wrapped stdout, rename stable tokens, or collapse semantically different exits.



OPERATING SYSTEM · CLI DESIGN

Commands for the human-led design passes

Each command mirrors a pass boundary and ends in its native machine token.

COMMAND	PASS	PURPOSE AND TERMINAL CONTRACT
`cvg capture`	0	Check a BRD or no-go artifact; `CHECK_BRD`.
`cvg intent`	1	Gate the technical requirements specification; `CHECK_TECH_SPEC`.
`cvg structure`	2	Scaffold or check ADRs and context; `CHECK_ADR`.
`cvg decompose`	3	Scaffold or check the swimlane plan tree; `CHECK_PLAN`.
`cvg review --adversary …`	4	Dispatch single or multi-engine refutation; `REVIEW`.
`cvg review --check`	4	Re-hash and gate objection closure; `CHECK_CONSENSUS`.
`cvg doctor`	Support	Report cross-family engine readiness; `DOCTOR`.
`cvg lane …`	Router	Classify FAST, NORMAL, or FULL; `LANE`.
`cvg next [--lane …]`	Conductor	Show the evidence board, next prompt, and closing gate; `NEXT_PASS`.

Discovery follows one consistent shape. Zero candidates is a usage error that names the searched locations. More than one candidate is also a usage error and lists the ambiguity. Exactly one candidate is passed to the gate. Explicit flags remain the safest choice in CI.

The design commands do not automate owner sign-off. They can scaffold, lint, attack, and establish freshness. A pipeline must still record the human decision at the barrier. Likewise, `cvg lane` suggests a path but does not run it. `cvg next` walks the chosen path but does not pass its gates. These omissions are intentional authority boundaries, not missing formatting.

Each command searches the artifact family it owns: `capture` and `intent` resolve `cvg/docs/brd/` and `cvg/docs/tech-spec/` first, then legacy flat prefixes and bare `docs/`; `structure` resolves `cvg/docs/adrs/` then `docs/adrs/`; `decompose` resolves `cvg/sketch/` then `sketch/`. Review consumes the current swimlane tree and writes its log under `sketch/.consensus/`. An explicit argument wins at every boundary.

Scaffold and check modes must not be confused. Draft modes establish structural progress without authorizing handoff. Final gates require canonical owner state and end in the pass token. Review dispatch can mutate the objection log; `review --check` is the freshness gate. In CI, invoke explicit paths and parse token plus exit code. In a human session, use discovery only after reviewing which candidate the command reports.

OPERATING SYSTEM · CLI BUILD

Commands for tasks, binding, execution, and proof

The build surface covers the full single-task lifecycle from authoring to a named settlement.

COMMAND FAMILY	CURRENT CAPABILITY
`cvg tasks new`	Create a Task-Spec through the shared backlog resolver and refresh derived state.
`cvg tasks validate [--no-state]`	Validate format, profile, sizing, and dependencies; refresh derived state unless `--no-state`.
`cvg tasks gate [--stamp]`	Run the PRE delegation gate; stamping is the explicit mutation.
`cvg tasks accept [--stamp]`	Run POST without writing Task-Spec/Converge metadata; `--stamp` writes acceptance.
`cvg tasks dod`	Render Definition of Done and behavior-to-eval traceability.
`cvg ready [--all]`	Filter `status: ready` plus dependencies done or legacy accepted; no leaf/PRE/precondition validation.
`cvg register [--check]`	Project or verify the optional tracker shadow.
`cvg bind [--check]`	Create or read-only recheck one execution profile and worker brief.
`cvg eval`	Run the Task-Spec's own eval suite.
`cvg verify`	Run tier-2 adversarial holdout review and record cross- or same-family independence.
`cvg loop --issue <id>`	Execute the bounded single-task loop and emit one terminal state.
`cvg transition`	Locked lifecycle move; `done` requires stamped acceptance and a matching pass receipt.

`cvg tasks metrics` queries append-only `created`, `accepted`, and `status_change` events. Loop attempts/checkpoints live separately under `cvg/loop/<id>/`. This shipped task-level command must not be confused with future program-level metrics or fleet management surfaces.

```
cvg transition T-20260729-export-format in-progress
cvg bind --task cvg/tasks/T-20260729-export-format.md
cvg loop --issue T-20260729-export-format
cvg tasks accept --stamp cvg/tasks/T-20260729-export-format.md
cvg transition T-20260729-export-format done
```

`ready` filters status plus dependencies that are done or legacy accepted; it does not validate leaf effort, PRE/trust, preconditions, or `blocked_reason`. `--all` restores the audit view. `cvg lint` owns cycle/overlap graph checks; dispatch policy is separate. `transition` locks, rewrites or moves the task, appends its event, then attempts a best-effort derived-index rebuild.

`eval` runs level-1 proof only; `verify` runs tier-2 holdout only; `tasks accept` runs POST only. None independently settles work or moves lifecycle. Full Loop coordinates attempts, level-1 eval, policy-selected tier 2, and settlement. Plain POST writes no Task-Spec or Converge metadata; only `--stamp` writes acceptance. A Loop success token without the matching pass receipt cannot enter `done`.

OPERATING SYSTEM · SETUP

Setup keeps credentials out of project state

Signing, tracker, repository, identity, projection, and runtime readiness are explicit and independently testable.

COMMAND	WHAT IT ESTABLISHES
`cvg init`	Create or detect the project workspace, seed typed document folders and the nine canonical conductor prompts, without confusing it with the installed tool root.
`cvg setup`	Present a readiness board for workspace, signing, engines, and optional integrations.
`cvg setup signing`	Provision a Git-private HMAC key with mode 0600; refuse unsafe or symlinked key paths.
`cvg setup key [linear|github|jira]`	Store a tracker secret in Keychain or a mode-0600 fallback outside the repository.
`cvg setup tracker`	Preflight the chosen backend and record non-secret adapter choices.
`cvg setup repo`	Derive a branch-pinned browsable source URL for issue backlinks.
`cvg setup identity`	Map backend or agent choices to tracker identities in machine-local private state.
`cvg setup projection`	Explicitly enable or disable higher-level tracker structure; disabled by default.
`cvg setup harness`	Scaffold the compact project router; never overwrite existing doctrine.
`cvg doctor runtime-contract`	Separately probe current-host controls; Bind/Loop do not consume this verdict automatically.

Configuration under `.cvg/` stores choices and caches, not tracker API secrets. `setup key` stores those outside the repository; signing setup separately provisions the Task-Spec HMAC key. External CLIs may also authenticate themselves. Atomic writes, private file modes, and symlink refusal protect local control state. Optional projection is disabled so a plain Register retains its established behavior until an owner opts into additional tracker structure.

Readiness tokens distinguish incomplete setup from unsafe continuation. Automation should consume those tokens and exact exit codes, not scrape the human-facing checklist.

`cvg setup` ends in `SETUP=READY|INCOMPLETE` and names one exact next step. Signing setup distinguishes a new key, an unchanged safe key, a non-Git project, an unsafe path, and usage error. Tracker setup can return `NEEDS_TEAM` when discovery is ambiguous and `UNREACHABLE` without persisting a broken choice. Repository setup returns `UNDETECTED` rather than fabricating a source URL.

Linear setup stores the resolved team identifier and never the API key. Identity mappings live in machine-local mode-0600 state and are excluded from Git; duplicate or invalid keys are refused atomically. Projection locks and caches use same-directory atomic writes and reject symlinks. Harness setup is non-clobbering: an existing router yields `PROPOSED` or `UNCHANGED`, not a rewritten project doctrine.

After any runtime, tracker, or repository change, rerun the relevant setup preflight and the consuming pass check. Setup readiness is a snapshot of the current environment, not a permanent certification.

OPERATING SYSTEM · AGENT PROTOCOL

One machine-readable contract for every command

Stable tokens, documented exit semantics, JSON, dry-run, and self-description make the CLI agent-native.

Most completed gates and Loop landings emit a stable token such as `CHECK_PLAN=OK`, `TIER=1`, `CHECK_VERIFY=REFUTED`, or `TASK_LOOP=EXHAUSTED`. Public gate-only RED and some Loop parser failures emit no `TASK_LOOP`; combine bounded output with exit code.

For routed operational commands, `--json` returns the envelope below. `agent-context` / `manifest` bypass it and emit native JSON; `help` / `version` bypass it and remain native text.

```
{
  "ok": false,
  "command": "verify",
  "token": "CHECK_VERIFY=REFUTED",
  "verdict": "REFUTED",
  "exit_code": 1,
  "changed": false,
  "dry_run": false,
  "data": {"output": "...CHECK_VERIFY=REFUTED"},
  "error": {
    "code": "REFUTED",
    "message": "CHECK_VERIFY=REFUTED"
  },
  "warnings": [],
  "meta": {}
}
```

The wrapper captures native command output. `cvg agent-context`, also exposed as `manifest`, publishes the command tree, advertised tokens, exit-code taxonomy, and global contracts in one JSON document. It is command-surface guidance; executable source and tests remain the semantic authority when summaries drift.

Loop JSON dry-run defect. In release 0.1.0, `--json --dry-run loop` drops `--dry-run` during wrapper reinvocation and may execute a real kernel or gate-only run. Use **text-mode** Loop dry-run. Also, Loop's top-level JSON token/verdict becomes the later `TRACKER=*` token; parse `data.output` for `TASK_LOOP=*`.

Parsing rule. Prefer JSON where the command's tested wrapper contract is sound. For Loop dry-run, use text mode. For non-dry Loop JSON, extract `TASK_LOOP` from captured output and combine it with exit code; tracker or cleanup output may follow.

`ok` reflects the documented command outcome, while `verdict` preserves the wrapper-selected semantic state. `changed` is reported, not an independent filesystem audit. `data.output` contains captured native text. In 0.1.0, `warnings` is always `[]` and is not populated from `WARN` lines; `error` and `meta` carry wrapper context.

`agent-context` publishes which exits are retryable and whether side effects may have occurred. An orchestrator should use that taxonomy rather than assuming every non-zero is safe to repeat. For example, a usage error is fixed at the caller; a stale profile is re-bound; a completed mutation followed by an outward integration failure requires reconciliation, not blind replay.

Parity tests should compare captured output and exit behavior command by command; the Loop exceptions above prevent treating the wrapper as universally lossless. Global flags remain syntax conveniences, not proof of semantic parity.

OPERATING SYSTEM · ARTIFACT MAP

What persists, what is derived, what is external

The workspace is an evidence graph, not a pile of interchangeable files.

```
cvg/
├── brain/
│   ├── _prompts/pass-0-*.md … pass-8-*.md
│   ├── transcripts/
│   └── decisions/
├── docs/
│   ├── brd/<slug>.md
│   ├── tech-spec/<slug>.md
│   ├── no-go/<slug>.md
│   ├── adrs/
│   ├── lessons/
│   └── CONTEXT.md
├── sketch/
│   ├── swimlane-*/
│   └── .consensus/objection-log.json
├── tasks/
│   ├── T-*.md
│   ├── queue/ done/ parked/
│   ├── _state.yaml
│   └── lifecycle ledger
├── execution/<task-id>/
│   ├── execution-profile.yaml
│   ├── AGENTS.task.md
│   └── guards + adapter manifests
├── loop/<task-id>/
│   ├── attempts/ + state.env
│   ├── tracker.log
│   ├── HANDOFF.md
│   └── STOP
└── receipts/
    ├── <task-id>.json
    └── <task-id>.attempt—<hash>.json
```

`brain/_prompts` is the conductor's seeded method surface: one prompt per pass from the pinned package. BRDs, specs, ADRs, plans, Task-Specs, and execution receipts are durable evidence. `_state.yaml` is a rebuildable index over canonical task frontmatter. `_metrics.jsonl` appends created, accepted, and status-change events; Loop attempt evidence lives elsewhere. Tracker issues and higher-level boards are external projections. A projection may be refreshed or reconstructed; it must never silently overwrite the sealed files from which it was derived.

Lifecycle statuses are `ready`, `in-progress`, `blocked`, `done`, and `parked`. An actual transition into `done` requires stamped `accepted: true` plus `cvg/receipts/<id>.json` with the same `task_id` and `result: pass`, not merely a textual status edit. A same-state `done → done` request returns NOOP before re-running that guard. Actual transitions rewrite/move atomically, append the event, then attempt a best-effort index rebuild whose failure is suppressed.

The artifact graph also defines the debugging route. A stale profile is re-bound; a malformed spec returns to Tasking; a missing decision returns to Design; a tracker mismatch is reconciled from Git truth.

CLASS	EXAMPLES	RECOVERY PROPERTY
Canonical authored truth	BRD, tech spec, ADRs, plans, Task-Specs	Changed only by the owning pass and renewed approval
Seeded method assets	`brain/_prompts/`	Refreshed from the pinned Converge package; never inferred from chat memory
Derived local state	`_state.yaml`, readiness views, execution profile	Rebuildable from canonical sources and static adapter/manifests; host doctor evidence is separate
Append-only history	Lifecycle/metrics ledger, Loop tracker log, `cvg/STATE.md`	Corrected by a later accounted event, not silent rewriting
Replaceable reviewed snapshot	Current objection log; latest execution receipt	Review dispatch overwrites the objection log. The receipt writer archives a differing local predecessor, but isolated sync may overwrite a differing destination without archiving it.
External projection	Tracker issues, links, PR, board hierarchy	Reconciled from canonical identity and settlement evidence

Moving a task into `done/` must not create a shadow `_state.yaml` inside that bucket. All index writers resolve the backlog root and scan lifecycle directories into one root index. Relative paths use the same anchor whether the layout is `tasks/` or `cvg/tasks/`. These details are architectural: a second state root can make the ready frontier and tracker projection lie while every individual file still looks valid.

PRACTICE · WORKED DESCENT I

Example: from operational pain to signed-off plan

A row-count observability gap shows how decisions become progressively more precise without jumping altitude.

Conducted route. The owner accepts FULL. `cvg next --lane FULL` points to Capture, and each green post-plus-gate pair advances the board through Intent, Structure, Decompose, and the Consensus barrier.

Capture. Operators report that a successful pipeline can publish incomplete output without an immediately visible count mismatch. The BRD quantifies investigation time, names the on-call user, defines the target detection window, and records the no-go comparison. The cost justifies action, so `CHECK_BRD` closes.

Intent. The technical spec requires every completed run to expose comparable input/output counts, define mismatch behavior, and emit an operator-visible signal within the target window. It explicitly excludes a redesign of unrelated data quality controls. `CHECK_TECH_SPEC` confirms measurable acceptance and traceability.

Structure. Live inspection identifies the canonical run record, the point at which counts become authoritative, and the alerting boundary. ADRs record those difficult-to-reverse choices; CONTEXT defines "input count," "published count," and "complete run." `CHECK_ADR` passes only against the grounded set.

Decompose. The plan cuts two real seams: capture and expose counts at the run boundary; evaluate and surface mismatch state. Each lane has a proving leg, and a steel thread demonstrates one run from recorded counts to visible mismatch. `CHECK_PLAN` closes.

Consensus. A different-family reviewer attacks partial failure, retry duplication, null counts, permissions, and a test that could pass while the operator-facing surface lies. Plans are repaired; one residual latency trade-off is accepted by the named owner. The log re-hashes cleanly, `CHECK_CONSENSUS=OK`, and the owner signs the barrier.

```
cvg capture cvg/docs/brd/row-counts.md
cvg intent cvg/docs/tech-spec/row-counts.md
cvg structure --final --dir cvg/docs/adrs
cvg decompose --dir cvg/sketch
cvg review --adversary codex,kimi
cvg review --check
```

The durable lineage might be: `docs/brd/row-counts.md` KPI "detect incomplete publication within five minutes" → requirement `FR-3` → ADR "completed-run record owns authoritative counts" → `swimlane-observability-leg-01` → Task-Spec behaviors `B-1..B-3`. Each identifier is a backlink, not a copy of the upstream content.

If live inspection shows counts are authoritative only after a different job than the brief assumed, Pass 2 records and reconciles that fact before planning. If the adversary discovers the operator surface can report healthy on unreadable state, the plan's proving strategy changes before any task is signed. The worked path is successful because it exposes correction early, not because every initial assumption survives.

PRACTICE · WORKED DESCENT II

Example: from Task-Spec to independently checked settlement

The build phase narrows one accepted leg into a bounded worker contract and a falsifiable terminal state.

The board crosses the barrier. After Consensus closes, `cvg next` returns Tasking. Register remains informational and opt-in; Bind and Loop stay mandatory for this FULL lane.

Tasking. The accepted count-capture leg becomes a runnable leaf. Its behaviors cover normal counts, missing count sources, and idempotent retry. Evals assert the output contract and exact failure token. Touch and create paths, dependencies, budgets, rollback, and an Exit Check are explicit. The PRE gate stamps a Tier-1 sign-off.

Register, if desired. The sealed spec projects to one issue; its dependency becomes one blocked-by link. The issue links to the canonical file. `CHECK_REGISTER=OK` proves board parity but does not redefine the task.

Bind. Pass 7 hashes the spec, relevant ADR, target module, and interface. It grants write access only to the named implementation and test paths, denies external writes, and statically validates manifests, topology, guards, and source freshness. The separate host doctor reports current-host readiness. The compact brief includes Goal, exact fences, Exit Check, assurance, and closure. `CHECK_RUNTIME_CONTRACT=PASS` closes static Bind.

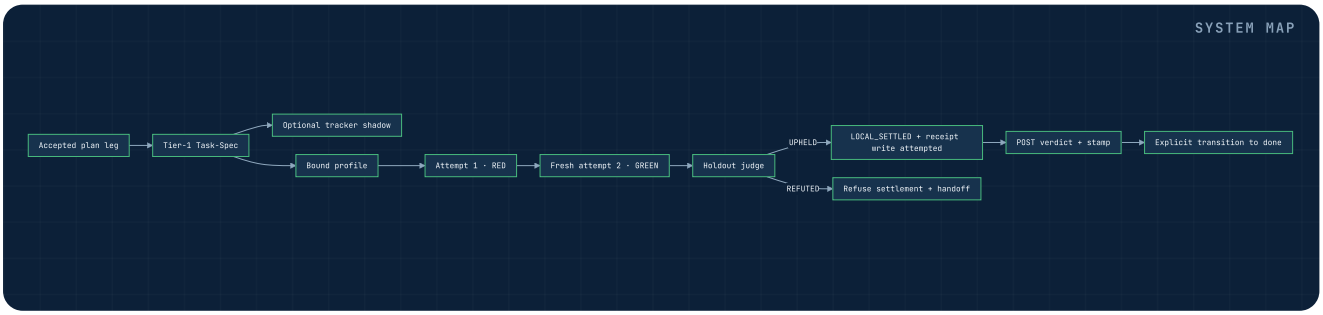
Loop. Attempt one writes the change and runs the sealed evals. A failure reveals incorrect retry behavior. A fresh second process reads the same sources plus exact failure output and corrects it. Level-1 evals turn green. The different-family judge checks a holdout about an unreadable count source. If upheld, path checks pass, the loop commits locally because external writes are denied, and reports `TASK_LOOP=LOCAL_SETTLED`. If refuted, the same green eval would not be enough; settlement would be refused.

The current receipt writer records schema, task/result, Task-Spec and profile paths with hashes, the eval-output hash and path-policy verdict, agent/branch, timestamp, generator, and optional post-hoc/bind-time provenance. The active `<task-id>.json` is the latest snapshot. The writer archives a differing predecessor in the workspace where it writes. Under isolated runs, the later sync copies the latest and attempt files home but does not archive a differing original-workspace destination before overwriting it. Preserve or commit destination receipt history before an isolated run.

```
cvg tasks gate --stamp cvg/tasks/T-20260729-export-format.md
cvg register --check                # only if projected
cvg doctor runtime-contract --runtime codex
cvg bind --task cvg/tasks/T-20260729-export-format.md
cvg loop --issue T-20260729-export-format
# CHECK_VERIFY=UPHELD
# TASK_LOOP=LOCAL_SETTLED
# confirm cvg/receipts/T-20260729-export-format.json says result: pass
cvg tasks accept --stamp cvg/tasks/T-20260729-export-format.md
cvg transition T-20260729-export-format done
```

Iteration count, budget spend, Tier-2 verdict, checked-path inventory, commit identity, and external-write policy are desirable stronger-conformance fields; the 0.1.0 writer does not encode them. Settlement calls the receipt writer best-effort, so a success token alone does not prove that the receipt exists. POST may run without the receipt; confirm a parseable matching `result: pass` receipt before relying on settlement and before the locked `done` transition. When `--post-hoc` encounters a changed Task-Spec hash, the writer requires Tier-1 sign-off and records `post_hoc` plus `sha256_at_bind`. If the hash still matches, 0.1.0 writes an ordinary receipt and does not attest post-hoc timing.

If registration is skipped, the repo-local task ID is supplied directly to the Loop's required `--issue` option; task truth does not change. If publication is allowed, the same scoped commit may progress to `SETTLED` after push plus an opened PR or printed manual PR handoff. In either case, POST stamping remains a separate referee step after Loop settlement, followed by the locked `done` transition. The difference between local and external settlement is policy and outward evidence, not code quality.



PRACTICE · RECOVERY

Route failures to the layer that owns them

Converge is fail-closed, but it is not dead-ended: every honest stop points to a repair altitude.

OBSERVED FAILURE	OWNING REPAIR
`CONDUCTOR_PRE=MISSING`	Return to the named earlier pass, produce its artifact, and run its authoritative gate.
The task's desired behavior is ambiguous	Return to Pass 1; renew downstream artifacts as needed.
A required architectural decision is absent	Pass 2 ADR and CONTEXT update, then re-plan and re-review.
A new system seam appears	Pass 3 decomposition and Pass 4 consensus.
Objection log hash differs from live plans	Re-run or reconcile Pass 4; obtain fresh owner sign-off.
Eval is syntactically broken or proves the wrong thing	Pass 5 repair and re-stamp; worker must not edit goalposts.
Tracker has duplicates or missing dependency edges	Reconcile Pass 6 projection from canonical specs.
Static profile is stale	Re-bind from current canonical sources.
Separate host doctor reports a missing control	Repair/select another host or record an allowed waiver before launch.
Same RED fingerprint repeats	Accept `STALLED`; diagnose the hypothesis before retrying.
Budget ends while progress exists	Use `HANDOFF.md`, decide whether to revise scope/budget, then explicitly resume.
Holdout refutes green work	The shipped Loop lands `BLOCKED`; repair, re-bind if needed, and explicitly resume as a separate action.

Do not repair a higher-altitude defect inside a lower-altitude artifact. That creates a fork in truth. When an upstream file changes, downstream hashes and approvals are intentionally invalidated. The cost of renewed gates is the price of maintaining causal evidence.

Failure tokens make automation safe, while blocked reports and handoffs make human recovery practical. A stop is valuable when it precisely identifies which assumption, input, control, or budget prevented trustworthy continuation.

Blocked report

Name the failed precondition, owning pass, last eval or verifier evidence, suspected upstream gap, and the smallest human decision required. Do not include speculative code as if it were the repair.

Current exhaustion handoff

`HANDOFF.md` records the stop reason, iteration/elapsed/optional token counters, the tail of the last attempt log, and generic next steps. It does not record diff or commit identity, the fingerprint value, a task-specific remaining hypothesis, or a reliably reconnecting resume command. Collect those items separately. To resume the retained working tree, operate there explicitly with `--isolation inplace --resume`; default isolation creates a fresh worktree.

In shipped Loop, kernel `--no-agent` RED, Tier-2 refusal, and settlement refusal land `BLOCKED`. Public `--gate-only` RED instead exits 1 with a report and no `TASK_LOOP`. Neither path is a dependency-readiness engine. `STALLED` means further identical attempts are irrational. `EXHAUSTED` means the agreed spend ended. `ERROR` means the mechanism itself could not operate safely. Those states may share a non-zero exit, but they demand different recovery and should never be collapsed into a generic retry queue.

After repair, renew invalidated proof in causal order: source artifact, semantic review, deterministic gate, signature, profile, then execution. A later green result cannot retroactively validate work performed under a stale epoch.

PRACTICE · RELEASE EVIDENCE

What release 0.1.0 has actually demonstrated

The package ships one version and evidence across all nine pass suites, the conductor, the single-task loop, board parity, and real holdout outcomes.

Release 0.1.0 is the first package release whose number is unified across the root, CLI, 12 skills, Task-Spec implementation, and manifests. A dedicated unity test fails on drift. All pass suites are wired into CI, including the Pass 4 barrier and the previously uncovered ADR gate.

The conductor adds a 20-check hermetic suite to that evidence. It proves a fresh FULL workspace starts at Pass 0, early starts are refused, artifacts move the evidence board in order, an unsigned Task-Spec does not close Tasking, Register remains opt-in, FAST starts at Tasking, and the sequence engine writes nothing. The twentieth regression proves legacy flat BRD and tech-spec files still count as evidence after the typed-folder migration. Those checks prove deterministic position and refusal behavior; the owning pass gates still prove artifact validity.

On the `uc-analytics` proving ground, nine of nine backlog tasks were swept: two landed `LOCAL_SETTLED`, seven landed `SETTLED` through merged PRs, and the final ready frontier was empty. Tracker parity reported nine specs to nine issues. This proves the tested workflow in that environment; it is not a claim that every runtime, repository, or integration has production history.

Tier 2 produced both meaningful outcomes. One different-family judge `UPHELD` a one-iteration implementation against its holdout. Another `REFUTED` green work because an exception path could report a false healthy state. The kernel refused settlement and preserved the blocked handoff. The independent seat therefore changed a real terminal result.

Known 0.1.0 gaps remain visible: Task-Spec's main skill file exceeds its own preferred length; lessons exist only for Passes 0–3; the verifier timeout is fixed by default; worktree resume does not restore the prior working tree; one task generator still has a known Git-root anchoring defect; and the refuted demonstration task intentionally remains blocked.

Evidence is strongest when bounded. These results establish that the current contracts can operate and refuse bad outcomes; they do not erase the documented gaps.

EVIDENCE	WHAT IT SUPPORTS	WHAT IT DOES NOT SUPPORT
All pass suites wired to CI	Every shipped suite is invoked and can reject covered negatives	Every semantic decision is correct
20-check conductor suite	Lane-aware order, prompt/gate handoff, typed-plus-legacy discovery, pre/post refusal, and read-only behavior	Artifact correctness or fleet scheduling
9/9 proving-ground sweep	End-to-end operation on that backlog and environment	Universal production reliability
9⇄9 registration parity	Faithful tracker mapping for the demonstrated adapter state	Tracker as canonical truth
UPHELD and REFUTED holdouts	Tier 2 can both allow and prevent settlement	Any judge is infallible

Reproduction begins with the release commit and root version, then runs the documented test or gate against named fixtures and workspace shapes. Logs and receipts are supporting artifacts, not substitutes for re-running cheap deterministic checks. Known gaps remain part of the release evidence because they constrain which environments and recovery promises can honestly be made.

PRACTICE · PRODUCT BOUNDARY

Shipped now, explicitly future

The current package is a complete pass chain for one assigned task, not yet a fleet operating system.

SHIPPED IN THE CURRENT PACKAGE	NOT YET BUILT
Passes 0–8 and their gate scripts	`cvg status` consolidated program view
12 skills: nine spine + three utilities	`cvg ci` consolidated CI orchestration
`cvg next` **evidence board, prompt handoff, and conductor pre/post hooks**	`cvg work` manager-facing work command
FAST/NORMAL/FULL lane classification	Fleet `run` / `route` dispatch
Task-Spec authoring, validation, PRE/POST, state, ledger	`board` / `graph` portfolio surfaces
Dependency-aware ready frontier	`deliver` and program-level delivery metrics
Optional tracker registration and parity check	Manager scheduling, fan-out, concurrency, fleet policy
Pass 7 static bind, plus separate host doctor, guards, brief	Standing autonomous multi-task operation
Single-task loop and eight terminal states	Cross-task resource and dependency arbitration
Optional Tier-2 verdict with recorded judge independence	Any command absent from the current command surface
Agent context, JSON envelope, dry-run	Automatic truth beyond the explicit gate contracts

`doctor`, `lane`, `next`, and `verify` are shipped. The remaining orchestration work is dispatch across a fleet, not classification or sequencing. Likewise, task-level lifecycle metrics are shipped; a broader delivery and portfolio layer remains roadmap.

The Manager is a future CI/CD concern outside the numbered chain. It will select among ready tasks and coordinate loops; it does not become a retroactive “Pass 7” or replace Bind. Documentation, automation, and demos should preserve this boundary.

Roadmap language test. “The ready frontier exists” is a shipped claim. “The system automatically selects and fans out that frontier” is future. “`cvg lane` classifies a route” is shipped. “`cvg next` identifies the next pass from artifact presence” is shipped. “`cvg route` dispatches a fleet” is future. “Task lifecycle metrics can be queried” is shipped. “Portfolio delivery analytics is complete” is future.

Future command names should not appear in tutorials as runnable examples or in diagrams without a clear roadmap style. Before adding an integration, query `cvg agent-context` from the installed package; if the command is absent, use the current lower-level contract or wait for the planned surface. Do not create a local wrapper with a future official name and then describe it as Converge behavior.

The current stop condition is precise: one assigned, signed, bound leaf can be driven to a named result. The conductor can walk the chosen descent to that leaf, but selection and coordination above the leaf remain an external human or CI responsibility.

PRACTICE · ADOPTION

Adopt Converge by increasing trust one boundary at a time

Start with durable artifacts and gates; earn unattended execution only after the evidence chain holds.

01 **Install and initialize.** Pin the package into a consuming repository, run `cvg init`, and verify `cvg version`.

02 **Establish signing.** Run `cvg setup signing`; keep the key Git-private and validate safe storage.

03 **Choose one real change.** Use `cvg lane` as a recommendation, then consciously select the route based on risk and existing evidence.

04 **Conduct one pass at a time.** Run `cvg next --lane ...`, use the returned prompt, require `pre N` before work, then pair `post N` with the authoritative pass gate before advancing.

05 **Close design at the right altitude.** Produce only the missing upstream artifacts. Run every applicable gate and record owner sign-off at Consensus.

06 **Author one excellent leaf.** Make behavior-to-eval traceability explicit, keep write scope small, and prove the PRE gate before delegation.

07 **Bind, then attest the host.** Run static Bind and the separate runtime doctor; review assurance levels and refuse hidden waivers.

08 **Begin locally.** Deny external writes, run the Loop, inspect eval output, terminal token, staged scope, handoff behavior, and receipt.

09 **Add independent proof.** Configure a different-family judge and verify that a refutation really prevents settlement.

10 **Project only if useful.** Enable Register after Git truth and the local frontier are reliable; gate parity after every adapter change.

Promotion criteria should be binary: required gates pass, no ambiguous discovery, no uncontrolled paths, receipts reproduce, and non-success states are handled rather than retried blindly. Scale the number of tasks only after the single-task evidence chain is boring.

PROMOTION	MINIMUM EVIDENCE
Draft → canonical design	Owner decision recorded; pass token successful; source paths unambiguous.
Canonical plan → machine phase	Fresh cross-family objection log, all dispositions closed, owner barrier signed.
Task → unattended	Tier 1, runnable leaf, complete DoD trace, dependency frontier clear.
Bound task → write	Current static epoch and hashes, plus separately reviewed host-doctor evidence.
Green → settlement	Scope recheck, required tier-2 verdict, policy-compliant commit/PR, and attempted receipt write; confirm the matching receipt before lifecycle close.

Run one negative drill before raising autonomy: tamper with a signed spec and observe Tier 3; introduce an out-of-scope file and observe settlement refusal; make the holdout refute visible-green work; create a STOP file and confirm `CANCELLED`. A system that only demonstrates its happy path has not shown that its brakes work.

Operational ownership should be named for signing keys, accepted waivers, blocked-task triage, tracker reconciliation, and release evidence. Automation without a human recovery owner merely moves ambiguity to the moment of failure.

REFERENCE · GLOSSARY

The vocabulary that keeps the chain coherent

Shared terms prevent a tracker, task, gate, and runtime from silently meaning different things.

TERM	CANONICAL MEANING
Barrier	Pass 4 Consensus plus explicit owner sign-off before machine-led build.
Conductor	The read-only sequence layer that derives the next lane pass from artifact presence, supplies its prompt, and enforces pre/post order without replacing the pass gate.
Gate	A deterministic check that proves a bounded contract and emits a stable token.
Lane	A FAST, NORMAL, or FULL route recommendation; not a pass or dispatcher.
Seam	A real system joint used to separate swimlane responsibility.
Leg	An independently provable stretch of one lane that yields Task-Spec leaves.
Task-Spec	A signed, vendor-neutral atomic work contract with runnable proof.
Tier 1	A valid keyed sign-off suitable for unattended delegation.
Epoch	One task ID plus one spec revision to which runtime authority is bound.
Capability envelope	Derived task-scoped rights plus declared prevention/detection assurance.
7A	The static, source-hash-bound execution profile, guards, and adapter manifests.
7B	The compact task-scoped brief: IDs, Goal, fences, Exit Check, boundaries, assurance, and closure.
Level 1	The Task-Spec's own visible eval and Exit Check.
Tier 2 verification	Adversarial second review against intent and a Holdout omitted from the brief but readable in the canonical spec; independence is recorded, not enforced.
Settlement	Git-visible-scope-checked green work committed locally or published under policy.
Ready frontier	`status: ready` tasks whose dependencies are done or legacy accepted; leaf/PRE/preconditions remain dispatcher checks.
Projection	A rebuildable external or derived view, never canonical task truth.
Manager	Future fleet selection and coordination outside the numbered pass chain.

Several pairs are deliberately distinct. A *gate* is a one-time check; the *Loop* acts, evaluates, and retries. A *tier* describes Task-Spec trust or independent verification context; an executor *conformance level* describes consumer behavior. A *lane* in the router is a ceremony route, while a *swimlane* is a system seam in the plan. A *tracker state* is a projection; Task-Spec frontmatter, stamped acceptance, the matching passing execution receipt, and the locked transition together establish lifecycle truth.

Likewise, *local settlement* is a complete success under a policy that forbids outward writes; it is not a partial failure. *Exhaustion* is a well-formed handoff but not success. *Detection* is an honest runtime assurance level but is weaker than prevention. *Waived* means accepted unenforced risk recorded in the profile, never that the requirement vanished.

Use these exact distinctions in issue text, dashboards, and automation. Flattening them produces false positives even when every individual command is working correctly.

REFERENCE · SOURCE MAP

Where to verify every contract

The handbook is an entry point; executable and skill sources remain the final authority.

QUESTION	AUTHORITATIVE SOURCE
Package release	`VERSION`, `CHANGELOG.md`, version-unity test
Pass ordering and method	`skills/README.md` and each pass's `SKILL.md`
Conductor order, prompts, and signals	`skills/conductor-steps/` skill, script, templates, and 20-check test suite
CLI commands and tokens	`./bin/cvg help`, `bin/README.md`, `cvg agent-context`
Task-Spec structure and gates	`skills/task-spec/` scripts, references, and tests
Registration semantics	`skills/task-specs-to-issues/`
Runtime profile and assurance	`skills/task-to-runtime-contract/`
Loop states and settlement	`skills/task-loop/`
Demonstrated outcomes and gaps	`CHANGELOG.md` under release 0.1.0
Future commands and Manager	`PLAN.md`, always labeled roadmap

Final operating principle. Converge does not ask anyone to trust a diagram, an agent, a board, or this PDF on reputation. Resolve the current artifact, run the current gate, inspect the current token and receipt, and keep the claim no broader than the evidence.

The method succeeds when a team can answer, for any settled change: which owner outcome justified it; which architecture and seams constrained it; which signed task authorized it; which evidence and capabilities were bound; which evals and holdout challenged it; which paths changed; and why the terminal state was allowed. That is convergence: agreement transformed into reproducible proof, with every stop and exception visible.

Fast verification sequence

Run `cvg version`; inspect `cvg agent-context`; resolve the canonical workspace; run the owning pass gate; recompute relevant hashes; inspect the terminal token and exit; trace the receipt back to its signed inputs.

Stop and investigate when

A command exists only in the plan, two roots contain competing state, a tracker differs from Git, an epoch is stale, a required control is merely asserted, a success lacks a receipt, or a non-success exit is presented as delivery.

For implementation detail, read the exact skill and script named in the source table, then its tests and negative fixtures. For release claims, read the current changelog entry together with known gaps. For future design, read `PLAN.md` but label every resulting statement as planned. This discipline keeps the handbook useful even as the package evolves: the map tells you where truth lives, while the executable source proves what is current.